

文内源码已迫于法务压力使用python重构，可访问：<https://github.com/hotjp/claude-code-cli>

Claude Code 模块级拆解

源码：<https://github.com/instructkr/claude-code>

副标题：从关键源码文件入手，拆开看它到底如何组织一个可运行、可扩展、可协同的 AI Agent 系统。

0. 怎么读这份拆解

这份文档只做一件事：

从关键源码文件出发，把 Claude Code 这套系统拆成若干可理解、可比较、可评价的模块。

阅读时可以先抓住三个层次：

1. 宿主层：程序如何启动、如何进入不同运行模式、如何托管会话
2. 运行时层：消息、状态、工具、权限、任务、远端连接如何协同
3. 协议与生态层：SDK、MCP、插件、技能、Hook、Remote 如何被统一到同一套结构里

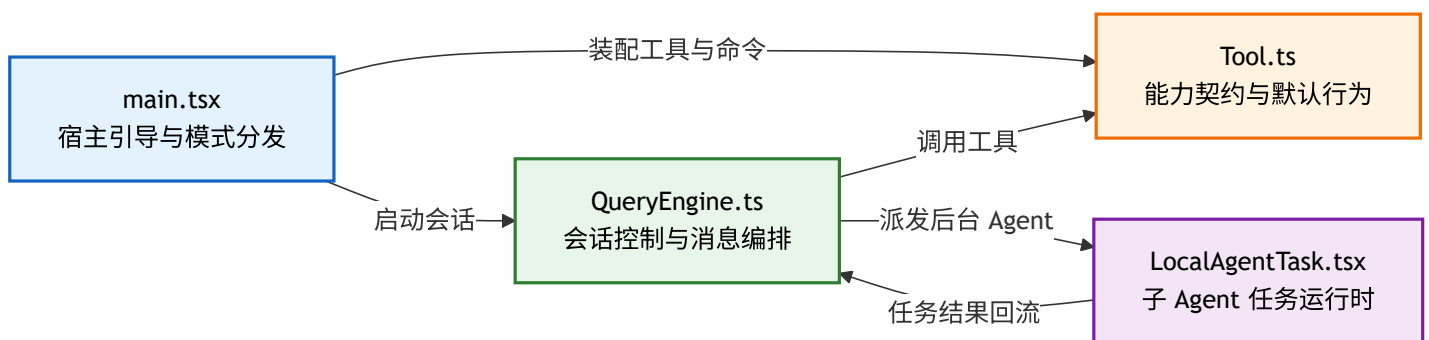
这意味着本文不是按目录机械罗列，而是按“系统为什么能成立”的主线往下拆。

最开始先抓四个支点就够了：

1. `src/main.tsx`
2. `src/Tool.ts`
3. `src/QueryEngine.ts`
4. `src/tasks/LocalAgentTask/LocalAgentTask.tsx`

它们分别代表：

- 启动与宿主装配
- 能力契约
- 会话运行时
- 异步子 agent 任务系统



如果只读目录，你会知道这系统“很大”。

如果把这四个文件读懂，你会知道它“为什么成立”。

0.1 整体结构先看什么

如果先不下钻到具体文件，Claude Code 可以先粗分成下面几块：

- 宿主引导层：负责启动、模式分发、初始化、桥接不同入口
- 命令与工具层：负责定义系统有哪些能力、如何被模型和用户调用
- 会话与状态层：负责消息流、上下文、运行时状态和 turn 生命周期
- 权限与沙箱层：负责把“能不能做”变成正式判定与 OS 级边界
- 模型接入层：负责 provider 适配、请求主循环、重试、fallback、预算控制
- 扩展生态层：负责 MCP、插件、技能、Hook、Remote 等外部能力接入

你可以把这份文档理解为：

Claude Code 的“活体模块地图 + 渐进式架构剖面图”。

0.2 重点模块总表

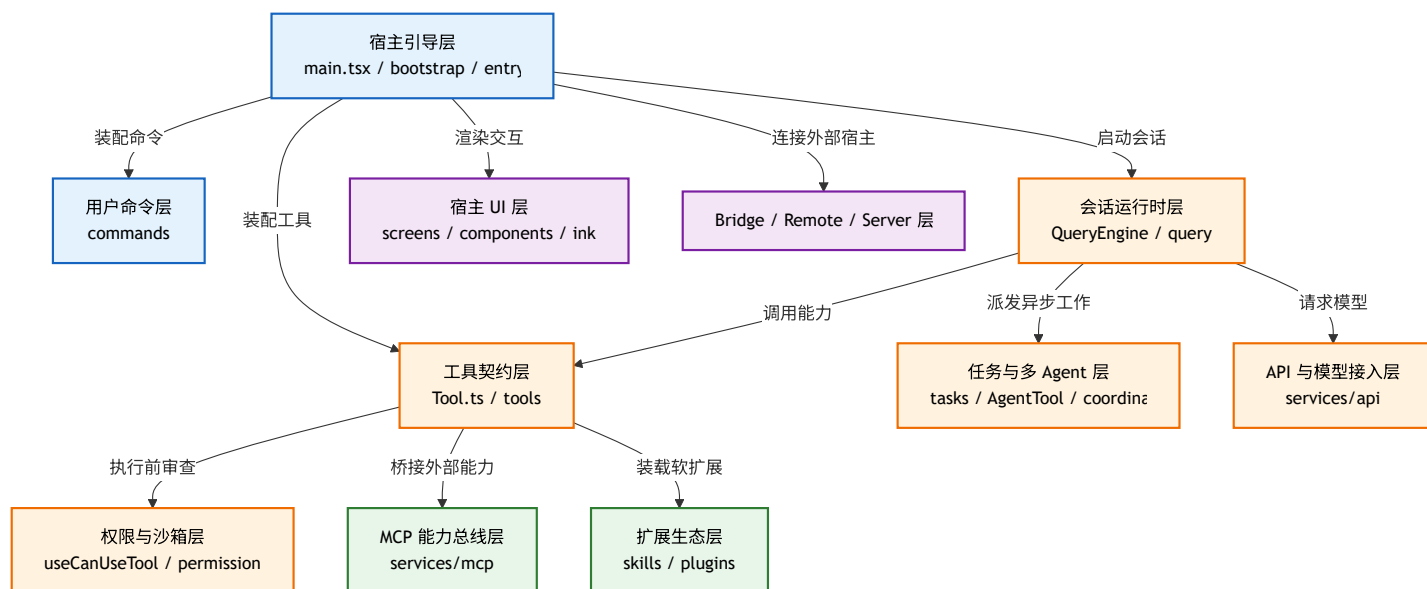
在继续逐个深拆之前，先把重点模块全景列出来。这样后面每新增一个章节，都能知道它在整套系统里的位置。

A 级核心模块

1. 宿主引导层： `main.tsx`、 `bootstrap/`、 `entrypoints/`
2. 用户命令层： `commands/`、 `commands.ts`
3. 工具契约层： `Tool.ts`、 `tools/`、 `tools.ts`
4. 会话运行时层： `QueryEngine.ts`、 `query.ts`、 `query/`
5. 状态层： `state/`
6. 任务与多 Agent 层： `tasks/`、 `coordinator/`、 `tools/AgentTool/`
7. 权限与沙箱层： `hooks/useCanUseTool.tsx`、 `utils/permissions/`、 `utils/sandbox/`
8. API 与模型接入层： `services/api/`
9. MCP 能力总线层： `services/mcp/`、 `tools/MCPTool/`
10. 扩展生态层： `skills/`、 `plugins/`
11. 宿主 UI 层： `screens/`、 `components/`、 `ink/`
12. Bridge / Remote / Server 层： `bridge/`、 `remote/`、 `server/`

B 级支撑模块

1. 上下文与记忆： `context/`、 `memdir/`
2. 配置演进： `schemas/`、 `migrations/`
3. 输入模式与交互增强： `keybindings/`、 `vim/`、 `voice/`
4. 类型与常量： `types/`、 `constants/`
5. 工程底座： `utils/`
6. 原生能力封装： `native-ts/`



这张图可以帮助你先建立一个总判断：

- `main.tsx` 不是普通入口，而是宿主装配器
- `Tool.ts` 和 `commands.ts` 决定 Claude Code 的能力边界
- `QueryEngine.ts` 与 `query/` 是会话运行时内核
- `services/api/` 决定它如何真正与模型世界打交道
- `skills/`、`plugins/`、`services/mcp/`、`remote/` 则决定它是否能长成平台

本文当前拆解顺序

为了让这份文档逐步长成完整地图，当前已经按这个顺序持续往下拆：

1. `main.tsx`
2. `Tool.ts`
3. `QueryEngine.ts`
4. `LocalAgentTask.tsx`
5. `AgentTool.tsx`
6. `useCanUseTool.tsx`
7. `services/mcp/client.ts`
8. `state/AppStateStore.ts`
9. `bridge/bridgeMain.ts`
10. `server/createDirectConnectSession.ts`
11. `utils/plugins/loadPluginCommands.ts`
12. `skills/loadSkillsDir.ts`
13. `server/directConnectManager.ts`
14. `utils/plugins/pluginLoader.ts`
15. `commands.ts`
16. `remote/RemoteSessionManager.ts`
17. `bootstrap/state.ts`
18. `services/api/claude.ts`

19. `entrypoints/sdk/controlSchemas.ts`
20. `query/stopHooks.ts`
21. `utils/sandbox/sandbox-adapter.ts`
22. `services/api/client.ts`
23. `remote/SessionsWebSocket.ts`
24. `query/tokenBudget.ts`
25. `entrypoints/sdk/coreSchemas.ts`
26. `query/config.ts`
27. `services/api/withRetry.ts`

1. `src/main.tsx`：不是入口文件，而是系统引导器

1.1 它真正负责的不是 `main()`，而是“把 Claude Code 这台机器启动起来”

这个文件最容易被低估，因为名字很普通。但它的实际职责非常重，几乎可以理解为 Claude Code 的 `bootloader`：

- 预热系统依赖
- 解析 CLI 模式
- 初始化设置与环境
- 运行迁移
- 接插件、技能、MCP、远程、bridge
- 初始化 REPL / server / ssh / direct-connect 等多种入口
- 把一堆 feature gate 编织进最终产品形态

这就是为什么它会极大。

1.2 开头的 `top-level side effects` 很“系统软件”

文件最前面做了几件明显偏性能工程的事：

- `profileCheckpoint`
- `startMdmRawRead()`
- `startKeychainPrefetch()`

这不是普通业务代码写法，而是在主动争夺启动关键路径。作者显然知道：

- 启动时间会影响产品体感
- 某些 IO 很慢，但可以和模块加载并行
- 一旦等到真正需要配置和认证时再读，就会把首屏拖慢

所以它选择在模块评估阶段就把慢操作打出去。

这背后体现的是一种很成熟的宿主思维：

CLI 不是脚本，而是产品；产品启动时间值得做专门优化。

1.3 runMigrations() 暴露了它是“长期演进系统”

src/main.tsx 里的 CURRENT_MIGRATION_VERSION 和 runMigrations() 很有代表性。这里做的不是数据库迁移，而是用户配置、模型字符串、开关状态等迁移。

这说明作者很清楚一件事：

Agent 产品的“兼容性”问题，不只是数据结构问题，也是模型命名、功能切换、权限模式、产品行为的兼容问题。

一个没有 migration 体系的 Agent 工具，很难长期迭代。

1.4 startDeferredPrefetches() 说明它把“首屏”与“首回合”区分开了

这个函数很值得夸。它把很多非首屏必需、但对首轮体验重要的东西延后：

- initUser()
- getUserContext()
- prefetchSystemContextIfSafe()
- prefetchOfficialMcpUrls()
- refreshModelCapabilities()
- 设置变更侦测器
- 技能变更侦测器

这背后其实是一套体验分层：

- 第一阶段：先把界面起来
- 第二阶段：在用户准备输入时偷偷预热
- 第三阶段：用户真正发第一轮时，大部分缓存已经在

这个设计很像浏览器、IDE 的启动优化思路，而不像普通 CLI。

1.5 main() 的本质是“模式分发器”

main() 自身做了很多分流判断：

- DIRECT_CONNECT
- deep link / URL scheme
- assistant

- ssh
- -p/--print
- sdk-url
- interactive / non-interactive

它不是单一路径启动，而是在统一宿主下分发出多个产品入口。

这意味着 Claude Code 并不是一个唯一运行模式的程序，而是一个统一 runtime 对多个前端入口的承载层。

1.6 run() 里 Commander 只是表层，真正重要的是 preAction

很多人读 CLI 会盯着 `.command()` 和 `.action()`。

但这个文件里更值得看的是 `program.hook('preAction', ...)`。

这里集中做了：

- 等待 MDM / keychain 预取完成
- `init()`
- 绑定 sink
- 注入 `--plugin-dir`
- 跑 migration
- 拉 remote managed settings
- 启动 settings sync

也就是说，作者把命令执行前的“公共基础设施准备”统一收口到了一个系统钩子里。这一点非常重要，因为它避免了每个子命令各自偷偷初始化一套环境。

1.7 这个文件的优点和代价

优点：

- 它把宿主初始化问题统一处理了
- 它对启动性能有明确优化意识
- 它把多入口模式统一在一个 runtime 下
- 它让插件、配置、权限、远程这些横切能力能在入口阶段介入

代价：

- 文件极其庞大，阅读门槛很高
- 各种 feature gate 和入口路径混在一起，心智负担大
- 它已经是“系统总线”，未来继续膨胀的风险很高

我的判断是：

`main.tsx` 的问题不是方向错，而是成功之后承担了太多。

2. `src/Tool.ts`：整个系统最重要的契约文件之一

2.1 它定义的不是工具函数，而是“能力协议”

很多 Agent 框架里的 Tool 定义大概只有：

- `name`
- `schema`
- `execute`

Claude Code 远远不止。

`src/Tool.ts` 把 Tool 定义成一个完整协议对象，至少包含这些层面：

- 调用： `call`
- 描述： `description`
- schema： `inputSchema` / `inputJSONSchema` / `outputSchema`
- 安全： `validateInput` / `checkPermissions`
- 行为属性： `isReadOnly` / `isDestructive` / `isConcurrencySafe`
- 模型暴露： `prompt`
- UI 渲染： `renderToolUseMessage` / `renderToolResultMessage` / `progress` / `error` / `rejected`
- 可观测性： `getActivityDescription` / `getToolUseSummary`
- 自动模式支持： `toAutoClassifierInput`
- 搜索与折叠： `isSearchOrReadCommand`

也就是说，这里的 Tool 不是“底层能力函数”，而是：

一个能被模型调用、能被权限系统审查、能被 UI 呈现、能被日志和摘要系统理解的运行时组件。

这才是真正能在产品里用起来的 Tool。

2.2 `ToolUseContext` 非常重，但重得有道理

这个文件里另一个核心对象是 `ToolUseContext`。它提供的不是几个简单参数，而是一整套宿主能力：

- `getAppState` / `setAppState`
- `setToolJSX`
- `appendSystemMessage`
- `sendOSNotification`
- `setInProgressToolUseIDs`

- `setResponseLength`
- `setStreamMode`
- `updateFileHistoryState`
- `updateAttributionState`
- `handleElicitation`
- `messages`
- `fileReadingLimits`
- `globLimits`

这说明作者接受了一个现实：

真正的工具调用不是纯函数，它发生在一个活着的宿主环境里。

它可能要：

- 改状态
- 发通知
- 显示临时 UI
- 往消息流里插系统消息
- 受当前 session 配置和限制影响

如果不把这些纳入协议，工具最终就会绕过架构，偷偷依赖全局单例。

2.3 它把“能力属性”显式化了

我很喜欢这里对能力属性的抽象，比如：

- `isReadOnly`
- `isDestructive`
- `interruptBehavior`
- `isOpenWorld`
- `requiresUserInteraction`
- `shouldDefer`
- `alwaysLoad`

这类字段的价值在于，它们把隐含语义变成了显式元数据。

这会直接影响：

- 权限策略
- 调度策略
- prompt 裁剪策略
- UI 呈现策略
- 自动模式判断

很多系统会等到逻辑长出来后，才发现这些语义散落在 `if` 里。Claude Code 是提前把它们对象化了。

2.4 buildTool() 是很小但很聪明的设计

`src/Tool.ts` 里的 `buildTool()` 并不复杂，但很好用。它提供统一默认值：

- `isEnabled`
- `isConcurrencySafe`
- `isReadOnly`
- `isDestructive`
- `checkPermissions`
- `toAutoClassifierInput`
- `userFacingName`

这个设计的妙处在于：

- 工具定义更简洁
- 默认策略统一
- 安全相关默认值倾向 `fail-closed`

特别是这些默认值里的态度很重要：

- 并发安全默认 `false`
- 只读默认 `false`
- `destructive` 默认 `false`
- `classifier` 输入默认空字符串

这说明作者不是只图方便，而是在用默认值表达平台立场。

2.5 它为什么重要

如果说 `main.tsx` 是系统引导器，`Tool.ts` 就是整个 Agent 世界与外部能力交界的宪法。

这份文件定义好了，工具生态才能长；
定义不好，后面每长一个工具都会破坏一致性。

我的评价是：

Claude Code 的 Tool 抽象是这份源码里最成熟、最“平台化”的设计之一。

3. src/QueryEngine.ts : 真正的会话运行时

3.1 它最重要的决定：把对话做成一个对象，而不是一个函数

QueryEngine 类一上来就把自己的定位说清楚了：

- 一次 conversation 一个 QueryEngine
- 每次 submitMessage() 是一个 turn
- messages 、 file cache 、 usage 、 permission denials 跨 turn 持续存在

这非常关键。

因为 Agent 的本质不是“发一次请求”，而是“在一段持续的上下文中工作”。

从架构角度看，这个类把以下东西聚合成了会话级状态：

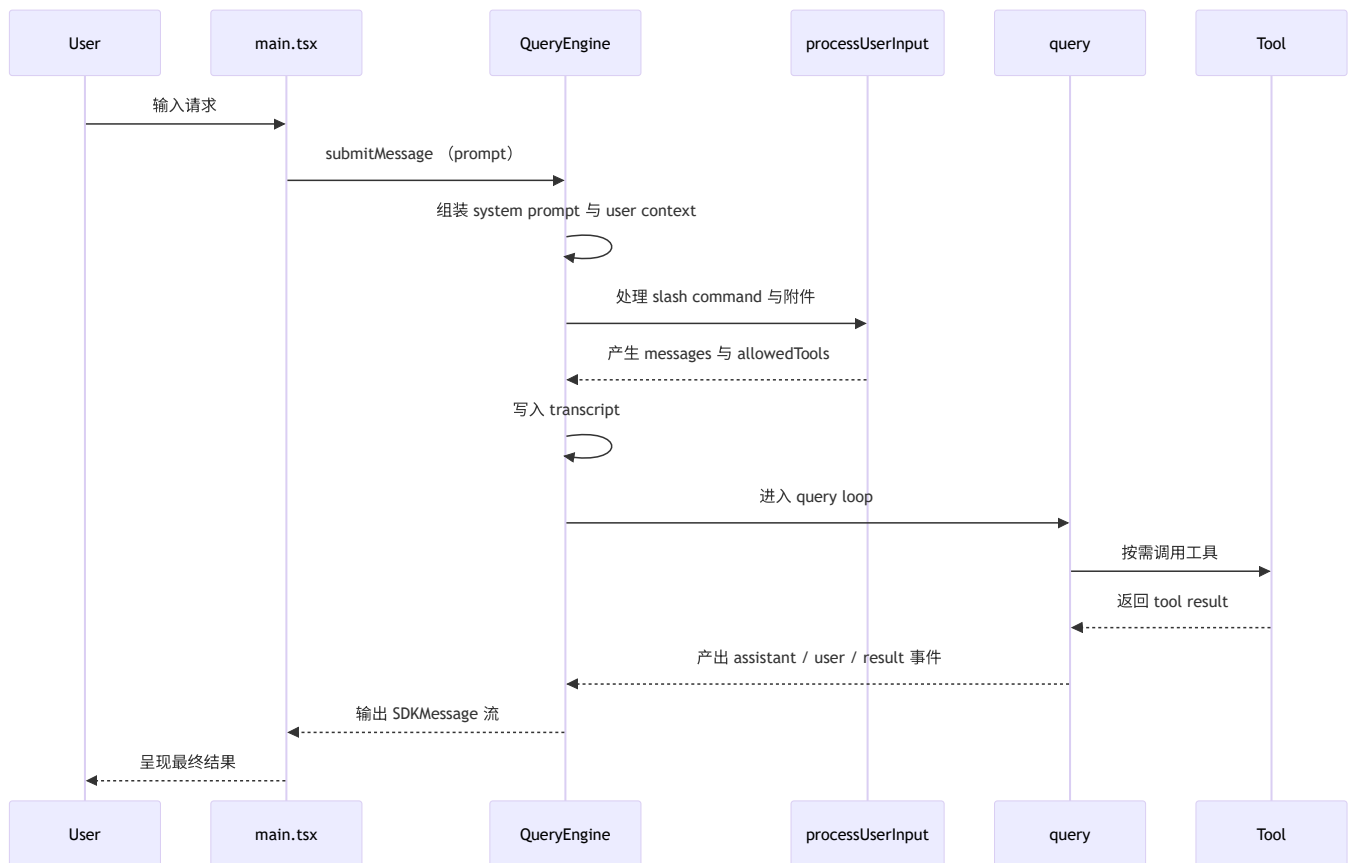
- mutableMessages
- abortController
- permissionDenials
- totalUsage
- readFileState
- discoveredSkillNames
- loadedNestedMemoryPaths

这正是一个运行时该拥有的状态。

3.2 submitMessage() 是会话级 orchestration，不只是发送 prompt

这个方法非常长，但结构其实很清晰。它做的主要是以下几件事：

1. 解析本轮 config
2. 包装 canUseTool ，记录权限拒绝
3. 计算初始模型、thinking 配置
4. 通过 fetchSystemPromptParts() 组装系统提示与上下文
5. 处理 memory prompt 注入
6. 构建 ProcessUserInputContext
7. 处理 orphaned permission
8. 跑 processUserInput()
9. 先写 transcript，再进入 query loop
10. 调用底层 query()
11. 累积 usage、回放消息、整理最终 result



这说明 QueryEngine 的职责并不是“替代 query.ts”，而是：

把底层 query loop 包装成一个面向会话的、能持久化和跨 turn 运作的服务对象。

换句话说：

- query.ts 更像单次 turn 的状态机
- QueryEngine.ts 更像多 turn session 的控制器

3.3 这里能看到 Claude Code 很强的“会话产品意识”

几个细节特别体现这种意识。

第一，先落 transcript 再跑 API

它会在真正进入 query loop 前先把用户消息写入 transcript。原因注释也写得很实在：

- 如果用户消息已经接受，但 API 还没响应时进程被杀掉
- 那么 resume 不应该丢失这条消息

这是非常真实的产品思维。

不是“功能逻辑上对不对”，而是“用户中途打断时体验会不会坏”。

第二，权限拒绝会被会话级追踪

wrappedCanUseTool 不是只为了这一轮判定，它会把 denial 存到 permissionDenials，最终回到 SDK result 里。

这意味着权限不是临时 UI，而是对话运行史的一部分。

第三，技能发现与嵌套 memory 路径会跨方法共享

像 `discoveredSkillNames`、`loadedNestedMemoryPaths` 这些看起来很细碎，但都在解决一个真实问题：

- 某些 `turn-scoped` 信息要在同一轮多个子过程之间共享
- 但又不能无限累积污染整个长期 `session`

这是典型的运行时状态治理问题。

3.4 类接口很克制，这反而是优点

`QueryEngine` 暴露的 `public surface` 很少：

- `submitMessage()`
- `interrupt()`
- `getMessages()`
- `getReadFileState()`
- `getSessionId()`
- `setModel()`

这说明作者没有把一堆内部状态操作直接暴露给外部，而是把“会话控制”的 API 保持得很窄。

我会把这看作一个好信号：

内部很复杂，但外部接口尽量收敛。

3.5 它的问题不是职责混乱，而是体量太大

我对这个文件最大的保留不是职责定义，而是规模：

- 逻辑高度集中
- 单方法很长
- 读起来需要同时记住很多上下文

但公平地说，这种复杂度不是写法不行，而是这个类真的在承担“`session runtime controller`”的责任。

所以它属于一种“合理但昂贵”的复杂。

4. src/tasks/LocalAgentTask/LocalAgentTask.tsx : 多 Agent 不是递归调用，而是任务系统

4.1 这个文件告诉你 Claude Code 对“子 Agent”真正的理解

如果只看宣传，很多人会把 sub-agent 理解为“再调一次模型”。

但看这个文件就知道，Claude Code 的理解更成熟：

子 agent 是一个后台任务对象。

它有：

- 独立 taskId
- 独立状态机
- 独立 output file
- 独立 abort controller
- 独立通知机制
- 前后台切换
- 进度追踪
- 结果摘要

这和“递归 function call”是两种完全不同的架构层级。

4.2 LocalAgentTaskState 很像一个微型进程控制块

它的字段非常多，但如果用操作系统视角看就很好理解：

- 标识：id、agentId、agentType
- 生命周期：status、startTime、endTime
- 控制：abortController、unregisterCleanup
- 输入输出：prompt、result、error、outputFile
- UI 管理：isBackgrounded、retain、diskLoaded
- 协同通信：pendingMessages
- 可观测性：progress、summary、lastReportedToolCount

它本质上就是一个被宿主管管理的子运行单元。

4.3 进度追踪设计很务实

这个文件前半段有一套 ProgressTracker 设计：

- toolUseCount
- latestInputTokens

- cumulativeOutputTokens
- recentActivities

并且会从 assistant message 里提取：

- 最近工具调用
- activity description
- 是否 search / read

这很聪明，因为它没有试图做“完美追踪”，而是抓住了用户真正关心的几件事：

- 它用了多少 token
- 它调了多少工具
- 它最近在干什么

这是很典型的产品化取舍：不追求全量内部状态可视化，而追求用户能理解当前进展。

4.4 enqueueAgentNotification() 是关键设计点

这个函数很重要，它把 task 完成事件编码成一个内部消息协议：

- <task-notification>
- <task-id>
- <status>
- <summary>
- <result>
- <usage>

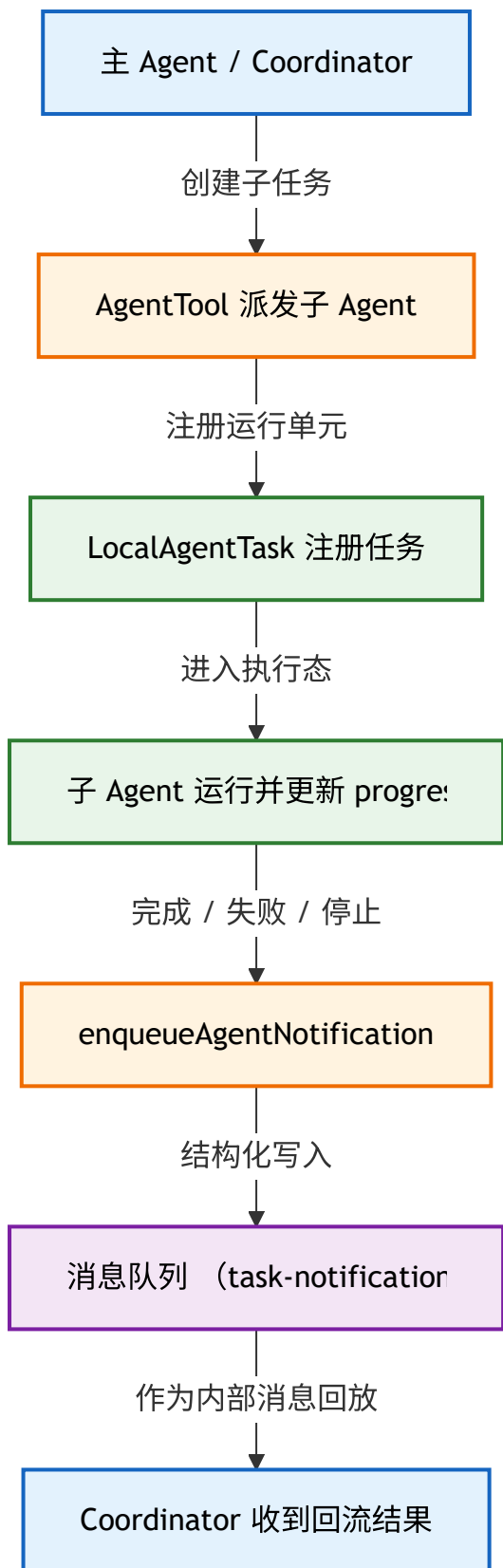
这说明 agent 结果返回给 coordinator 的方式，不是共享内存，不是直接函数回调，而是：

转成消息流中的结构化事件。

这有几个巨大好处：

- 主线程可以统一按“消息”来处理异步结果
- UI 和 SDK 都能消费同一套协议
- agent 完成与用户发消息共享一条事件总线
- 恢复、记录、显示都更统一

这就是很典型的 event-driven runtime 思路。



4.5 前台/后台切换体现了真实交互需求

这个文件不只支持背景任务，还支持“前台跑一会儿，再后台化”：

- `registerAgentForeground()`
- `backgroundAgentTask()`
- `unregisterAgentForeground()`

这说明作者认真处理了一个真实场景：

- 用户刚发起子 agent 时，想先看看它在做什么
- 如果它跑太久，就应该允许退到后台
- 后台之后还要能通知、恢复、查看 transcript

这不是一般 demo 会处理的问题，但是真正产品一定会碰到。

4.6 它把“取消”也当成一等能力

`killAsyncAgent()` 和 `killAllRunningAgentTasks()` 看起来普通，其实很关键。一个多 agent 系统如果不能稳定取消，就不可能可控。

这里的取消不是“设置个标志位”就完事，而是涉及：

- `abortController.abort()`
- cleanup 注销
- 状态切换为 `killed`
- 结果文件驱逐
- UI 宽限时间 `PANEL_GRACE_MS`

说明作者把取消操作当成真正的生命周期转换在设计。

4.7 这个模块最值得借鉴的地方

如果你在做多 Agent 产品，我觉得这个文件带来的最大启发是：

子 agent 的核心问题不是“怎么生成”，而是“怎么管理”。

生成一个子 agent 很容易。

难的是：

- 如何跟踪
- 如何取消
- 如何通知
- 如何显示
- 如何恢复
- 如何避免前后台状态错乱

Claude Code 在这方面是认真做过工程的。

5. src/tools/AgentTool/AgentTool.tsx：多 Agent 系统的总入口

如果说 src/tasks/LocalAgentTask/LocalAgentTask.tsx 负责“把子 agent 当任务管理”，那么 src/tools/AgentTool/AgentTool.tsx 负责的就是另一半：

决定这次子 agent 究竟应该以什么形态被启动。

这是 Claude Code 多 Agent 架构里非常关键的一层。

因为真正困难的地方，从来不是“能不能起一个 agent”，而是：

- 起哪个 agent
- 它有没有资格起
- 是同步跑还是异步跑
- 是本地跑还是远程跑
- 要不要隔离工作目录
- 结果应该怎样重新编码回主线程

这些都不是 runAgent() 一个人能解决的。

AgentTool.tsx 做的，是把“调用子 agent”这件事正式提升为一个运行时调度问题。

5.1 它在系统里的位置，不是普通 Tool，而是 Agent 路由器

这个文件表面上也是 buildTool({...}) 出来的一个 tool，但它和普通工具不一样。

普通工具通常只是：

- 定义输入
- 执行逻辑
- 返回结果

而 AgentTool 还额外负责：

- 从当前工具池反推出可用 MCP server
- 从权限规则里筛掉不可用 agent
- 从 coordinator / assistant / proactive 等模式决定执行形态
- 把工作交给 spawnTeammate()、teleportToRemote()、registerAsyncAgent()、runAgent()
- 最后再把不同执行路径的结果统一映射成 tool result

所以它更像：

Claude Code 多 Agent 能力的入口编排层。

5.2 它的输入 schema，本身就在暴露架构设计

这个文件的输入参数非常值得看，因为参数本身已经透露出作者脑中对“子 agent”是什么的理解。

基础参数有：

- description
- prompt
- subagent_type
- model
- run_in_background

多 Agent / swarms 相关参数有：

- name
- team_name
- mode

隔离与执行环境参数有：

- isolation
- cwd

这说明作者定义的不是“调用一个模型”，而是“生成一个有身份、有运行方式、有工作空间的执行单元”。

你可以把这套 schema 理解为三层：

1. 任务层：要做什么
2. 身份层：由谁做、能否被继续寻址
3. 环境层：在哪里做、怎样隔离

很多项目做到第一层就停了，Claude Code 明显做到了第三层。

5.3 prompt() 暴露了一个重要事实：可用 agent 不是静态列表

prompt() 这段逻辑很成熟。

它不是简单把全部 agent 名字拼进 prompt，而是先做两层过滤：

1. 根据当前工具池里有哪些 mcp__... 工具，抽出当前真正可用的 MCP server
2. 用 filterAgentsByMcpRequirements() 和 filterDeniedAgents() 过滤 agent

这背后的设计含义很重要：

一个 agent 是否可选，不只取决于“有没有定义”，还取决于它当前是不是有足够的外部能力和权限。

这意味着 Claude Code 的 agent catalog 不是静态技能列表，而是：

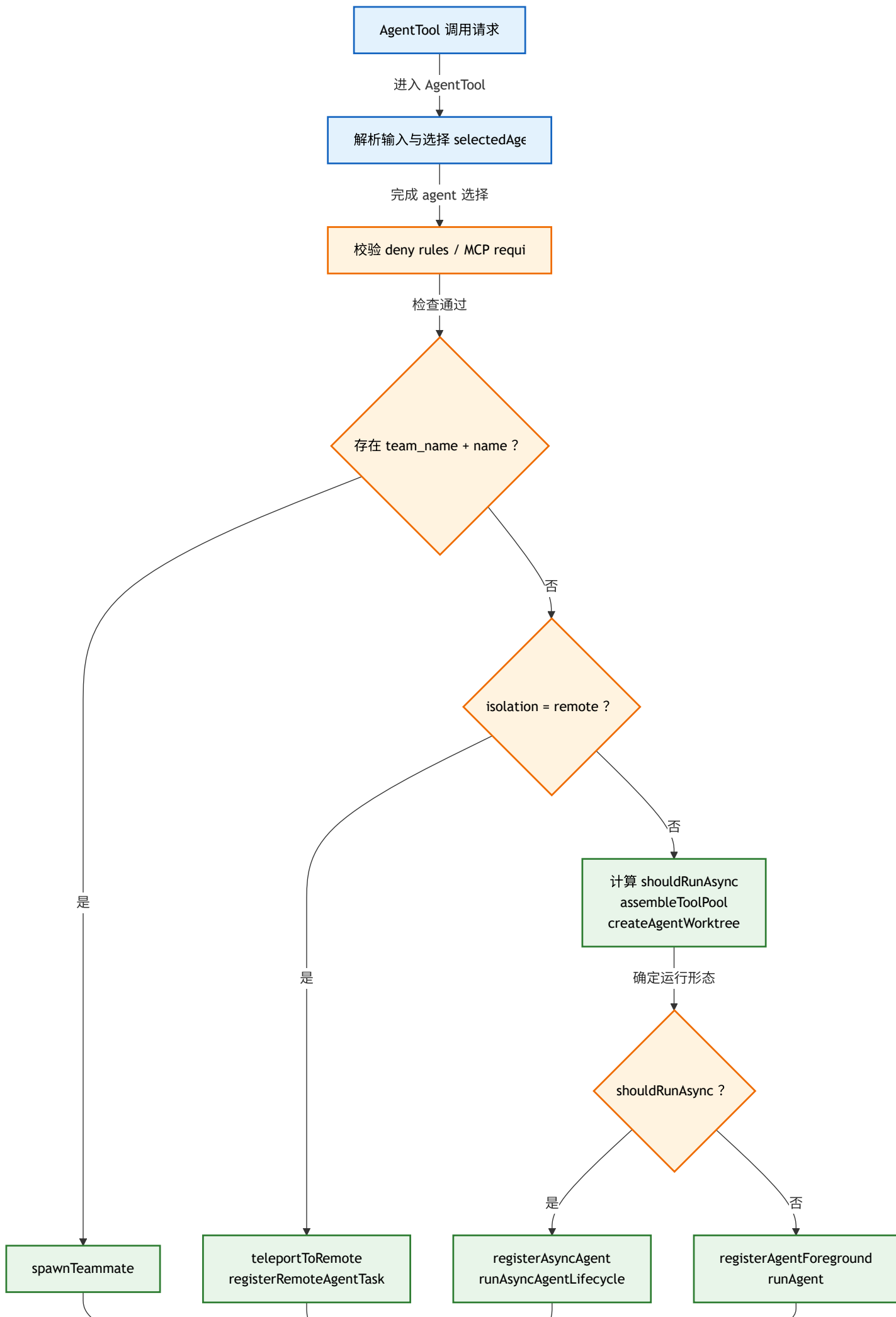
带运行时约束的可选角色集合。

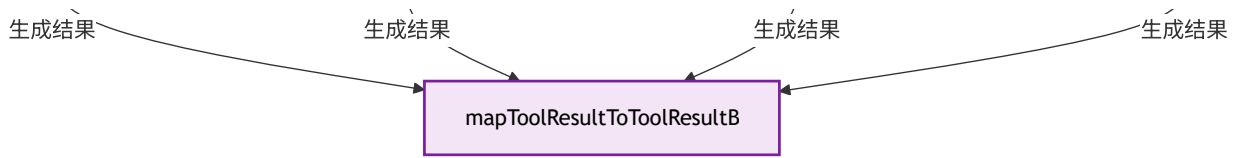
这比很多“agent registry”实现都更像真产品，因为它防止模型选到一个名义上存在、实际上跑不起来的 agent。

5.4 call() 的真正作用，是做执行形态分流

这个文件最精彩的地方就在 `call()` 。

它不是一条执行链，而是一颗分流树。





这棵树至少有四种主要执行形态：

1. teammate / team spawn
2. remote launch
3. async background agent
4. sync foreground agent

这就说明 AgentTool 并不是“一个工具”，而是：

把多个子系统编排成统一入口的 dispatcher。

5.5 teammate 路径说明 Claude Code 不只做单会话多 agent

当 `team_name` 和 `name` 同时存在时，AgentTool 不走普通 subagent 路径，而是直接调用 `spawnTeammate()`。

这里很有意思，因为它说明 Claude Code 区分了两类东西：

- subagent：作为当前会话内部的工作单元
- teammate：作为 team 上下文里的可命名成员

而且这个文件还专门处理了一些团队模式下的边界条件：

- teammate 不能再递归地产生 teammate
- in-process teammate 不能再起 background agent
- plan mode 可以通过 `mode` 传入

这说明作者对“多 Agent 协同”并不是只停留在技术层，而是已经开始做：

- 组织结构约束
- 成员可寻址性
- 生命周期边界

这很像真正的协作系统，而不是 demo 里的“多开几个模型”。

5.6 remote 路径说明它已经把执行环境抽象成可切换后端

当 `effectiveIsolation === 'remote'` 时，这里会走：

- `checkRemoteAgentEligibility()`
- `teleportToRemote()`
- `registerRemoteAgentTask()`

然后返回 `status: 'remote_launched'`。

这件事的价值非常大，因为它说明 `AgentTool` 调度的不是某个固定运行时，而是一个：

可本地、可远程的 Agent execution backend。

换句话说，在 Claude Code 的抽象里，“起一个 agent”已经不等于“在当前进程里递归调用一次模型”。它可以被迁移到另一种执行宿主里。

这也是为什么我会说这套架构更接近 runtime，而不是 prompt wrapper。

5.7 `shouldRunAsync` 的计算非常像产品逻辑，而不是技术逻辑

这里有一个很能体现产品成熟度的判断：`shouldRunAsync` 不是简单等于 `run_in_background`。

它同时受这些因素影响：

- 用户是否显式传了 `run_in_background`
- agent definition 里是否配置了 `background: true`
- 当前是否处于 coordinator mode
- fork subagent 实验是否开启
- assistant mode 是否开启
- proactive mode 是否激活
- 全局是否禁用了 background tasks

这说明后台化不是一个单点功能，而是一种系统级策略。

也就是说，在 Claude Code 眼里：

agent 是否应该后台运行，是调度器根据交互模式和系统模式共同决定的。

这就比“前端传个 `background=true`”要成熟很多。

5.8 它把 worker 的工具池重新组装，而不是继承父 agent 的全部能力

这一段我很喜欢：

- `workerPermissionContext`
- `assembleToolPool(...)`

作者没有让子 agent 直接复用父 agent 的工具快照，而是重新根据 worker 自己的 permission mode 组装工具池。

这件事很关键，因为它避免了两个常见问题：

1. 父 agent 的临时限制意外污染子 agent
2. 子 agent 的权限边界不清晰

这说明 Claude Code 对 agent 的理解不是“一个函数套一个函数”，而是：

一个有自己工具视角、自己权限视角的执行主体。

5.9 worktree 和 cwd 处理说明作者真的在意工程隔离

这个文件对执行目录的处理也很成熟。

核心点有两个：

- 显式 cwd override
- worktree 隔离

当启用 worktree 时，它会：

- 提前生成 agentId
- 用 createAgentWorktree() 建独立工作目录
- 在 fork path 下附加 worktree notice，提醒子 agent 重读文件并做路径转换
- 结束时通过 cleanupWorktreeIfNeeded() 决定是否清理

而 cleanupWorktreeIfNeeded() 也不是无脑删：

- 如果没有改动，就自动移除 worktree
- 如果有改动，就保留 worktree 并把路径写回元数据
- 如果是 hook-based worktree，也倾向保留

这说明作者非常清楚一个现实：

隔离的目标不是“用完即删”，而是“既避免污染主工作区，也不丢失有效修改”。

这是很有工程感的取舍。

5.10 sync 路径最有意思的地方，是它支持“运行中转后台”

如果只看异步分支，你会以为同步和异步是启动前就决定的。

但这里真正高级的地方在于：

- sync agent 会先 registerAgentForeground()
- 然后在执行循环里监听 backgroundSignal
- 一旦触发，就把当前前台运行转换成后台继续执行

也就是说，这里支持的不是单纯：

- 要么前台
- 要么后台

而是：

先前台观察，再根据运行时态势切换到后台。

这很像真实产品场景。

因为用户刚起一个 agent 时，往往想先看两眼；如果它开始长跑，就应该允许它退到后台，等完成后
再通知。

这类“中途迁移生命周期”的设计，通常只有真正做过长期交互系统的人才会补。

5.11 权限处理说明 AgentTool 在 auto mode 下被当成高风险入口

checkPermissions() 的逻辑很简洁，但意味很强：

- 非 auto mode: allow
- auto mode: passthrough

这相当于承认了一件事：

spawn subagent 不是普通工具调用，而是高阶动作。

因为一旦允许它自动发生，后面就不是一次工具执行，而是一整段新的行动链。

所以作者选择在 auto mode 下把它交给更严格的审批路径，而不是跟普通工具一视同仁。
这个决策我非常认同，因为它抓住了多 Agent 系统里真正危险的地方：

不是单个 bash，而是无边界扩张。

5.12 mapToolResultToToolResultBlockParam() 负责把多种运行形态重新压平

最后这个函数非常关键。

因为前面分了很多执行路径，但回到主 agent 时，还是得回到统一的 tool_result 语义里。

这里至少处理了四类结果：

- teammate_spawned
- remote_launched
- async_launched
- completed

它做的事情不是简单 stringify，而是：

- 为不同状态生成不同提示文案
- 告诉主 agent 什么时候应该结束响应
- 告诉主 agent 什么时候不要与后台 agent 重复劳动
- 在完成态里附加 agentId 、 usage 、 worktreePath

这说明作者清楚一个重要事实：

真正难的不是 spawn，而是让上层 coordinator 正确理解 spawn 的后果。

所以这个映射函数本质上是：

运行时状态到 LLM 可消费协议的翻译器。

5.13 这个模块最值得借鉴的地方

如果让我总结 `src/tools/AgentTool/AgentTool.tsx` 最值得学的点，我会给三个关键词：

1. 分流
2. 隔离
3. 压平

分流，是把不同子 agent 形态统一放进一个入口。

隔离，是给不同 agent 独立的工具池、权限和工作目录。

压平，是把复杂运行结果重新折叠成 coordinator 能理解的消息协议。

所以这个文件真正厉害的地方，不是“能起子 agent”，而是：

它把子 agent 从一个技巧，做成了一种系统能力。

6. `src/hooks/useCanUseTool.tsx`：权限系统真正落地的编排入口

这个文件的名字很容易让人低估它。

看起来它像一个普通 React hook，仿佛只是：

- 调一下 `hasPermissionsToUseTool()`
- 不行就弹个确认框

但实际不是。

真正读完 `src/hooks/useCanUseTool.tsx` 以及它依赖的：

- `src/hooks/toolPermission/PermissionContext.ts`
- `src/hooks/toolPermission/handlers/interactiveHandler.ts`

- `src/hooks/toolPermission/handlers/coordinatorHandler.ts`
- `src/hooks/toolPermission/handlers/swarmWorkerHandler.ts`

你会发现它干的事情是：

把权限判断从一个布尔检查，做成了一个可编排、可竞争、可回退的异步决策流程。

6.1 先说结论：它不是权限规则引擎，而是权限决策编排器

Claude Code 的权限系统不是都写在这个 hook 里。

真正的规则判断在：

- `hasPermissionsToUseTool(...)`
- permission hooks
- bash classifier
- bridge / channel 远端审批
- swarm leader 回信

而 `useCanUseTool.tsx` 负责的是把这些来源统一起来。

所以它在架构里的位置更准确地说是：

Permission Orchestrator。

它统一处理：

- 权限决策的入口
- 决策结果的归一化
- 决策竞争时的 winner-takes-all
- UI 队列与 AppState 更新
- abort/cancel 的一致收口

这类模块通常比“规则本身”还重要，因为真实产品里最容易乱的不是规则，而是：

- 多种规则同时存在
- 多个审批来源同时返回
- 用户在过程中取消
- 自动检查与人工确认交叉发生

而这个文件正是在收拾这些复杂性。

6.2 它返回的不是布尔值，而是完整的 `PermissionDecision`

这个 hook 暴露出来的是 `CanUseToolFn`，返回的是 `Promise<PermissionDecision<Input>>`。这很关键。

因为这说明在 Claude Code 里，权限判断并不是：

- `true`
- `false`

而是至少三态：

- `allow`
- `deny`
- `ask`

而且还会带上：

- `updatedInput`
- `decisionReason`
- `acceptFeedback`
- `contentBlocks`

这说明作者理解的“权限”不是单纯的门禁，而是：

一次可能修改输入、附带解释、甚至带回额外上下文的交互式决策。

6.3 主函数虽然短，但实际是在建立一场异步决策竞赛

`useCanUseTool()` 本体很短，甚至还带着 React Compiler 的 `_c(...)` 痕迹，看起来有点像编译产物。但逻辑主线非常清晰：

1. 建 `PermissionContext`
2. 先检查 `abort`
3. 生成 `decisionPromise`
4. 对 `allow / deny / ask` 三类结果分别分流
5. 在 `catch/finally` 里统一收尾

这里最值得注意的是，它把整个过程包在一个 `new Promise(resolve => ...)` 里。这不是坏味道，反而说明作者知道这不是一个单 `await` 就能结束的流程。

因为后面的决策可能来自：

- 本地规则
- `classifier`
- `hook`
- UI 点击
- `bridge` 回复
- `swarm leader` 回复

这些来源都可能异步地、在不同时间点回来。
所以这里必须有一个总 `resolve` 来承接最终赢家。

6.4 `PermissionContext` 是这个权限系统真正的地基

如果只看 `hook` 主体，你会觉得它有点像 `if/else` 总线。
但一旦看 `src/hooks/toolPermission/PermissionContext.ts`，味道就出来了。

这个 `context` 不是简单传参对象，而是把权限流程常用能力都封装起来了：

- `logDecision()`
- `logCancelled()`
- `persistPermissions()`
- `resolveIfAborted()`
- `cancelAndAbort()`
- `tryClassifier()`
- `runHooks()`
- `buildAllow()`
- `buildDeny()`
- `handleUserAllow()`
- `handleHookAllow()`

这说明作者不是在“页面里顺手写权限流程”，而是在抽一个专门的权限执行上下文。

这类设计的价值在于：

- 不同 `handler` 复用同一套能力
- 所有分支的日志与持久化行为统一
- `reject / abort / allow` 的输出格式统一

也就是说，真正的一致性不是写在 `useCanUseTool.tsx` 里，而是写在 `PermissionContext` 里。

6.5 `allow` 分支很简单，但已经埋进了 `classifier` 与审计语义

如果 `hasPermissionsToUseTool()` 直接给出 `allow`，这个 `hook` 会很快放行。
但它不是纯粹地 `resolve(result)`，而是还做了两件事：

- 记录 `auto-mode classifier approval`
- 用 `ctx.logDecision()` 记一条接受决策

说明即使“自动放行”这种看似最简单的路径，也被作者当成：

需要被解释、被追踪的系统行为。

这很重要，因为 `Agent` 产品一旦进入 `auto mode`，最怕的就是：

- 工具为什么被放行说不清
- 事后无法追溯

Claude Code 在这点上明显是有产品意识的。

6.6 deny 分支也不是终点，它还要服务 UI 和记忆纠偏

deny 分支也不只是拒绝。

这里除了 `logPermissionDecision(...)`，还会在 auto mode classifier 拒绝时：

- `recordAutoModeDenial(...)`
- `addNotification(...)`

再结合 `PermissionContext.cancelAndAbort()` 里对：

- `REJECT_MESSAGE`
- `SUBAGENT_REJECT_MESSAGE`
- `withMemoryCorrectionHint(...)`

的处理，可以看出作者在处理一个很细的问题：

被拒绝这件事，不只是给程序一个状态，也要给模型和用户一个后续可行动的解释。

也就是说，Claude Code 的 reject 不是“黑盒失败”，而是：

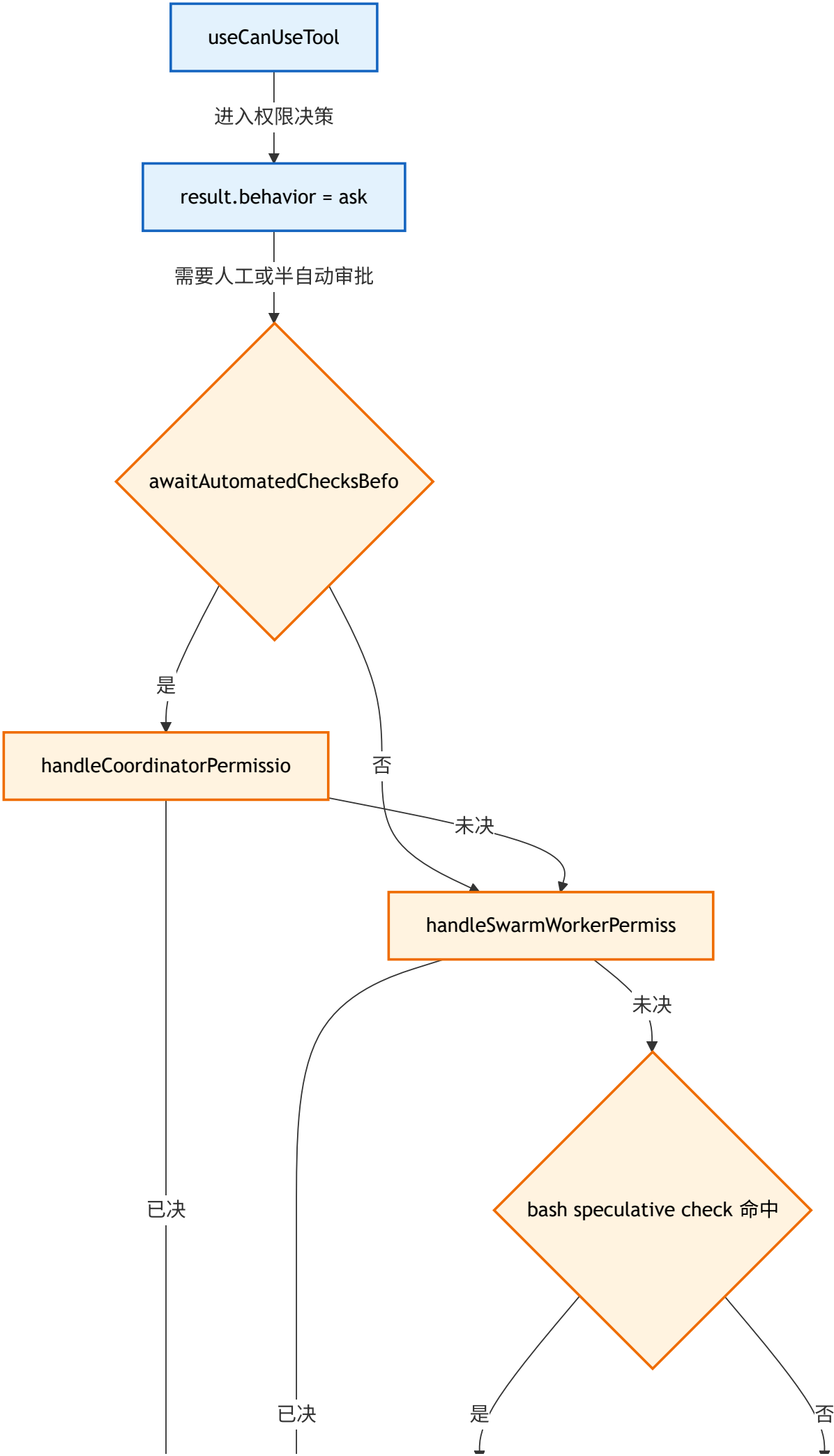
- 对系统：有日志
- 对用户：有通知
- 对模型：有纠偏提示

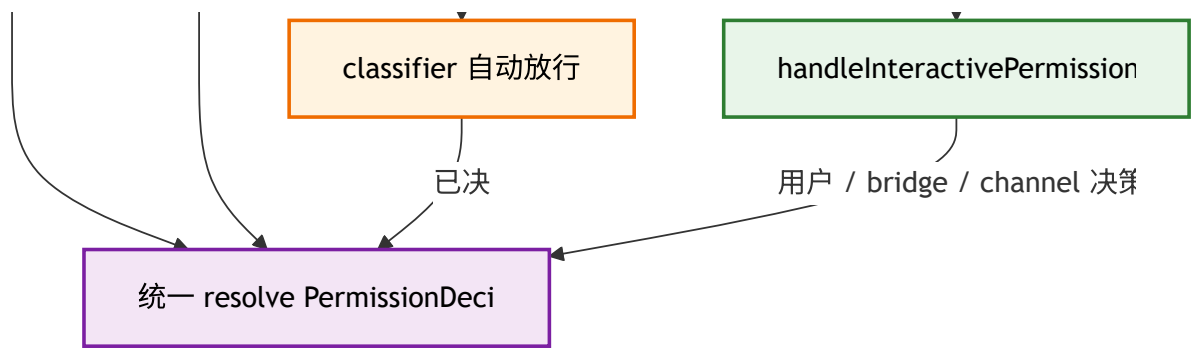
这个体验层设计做得是挺到位的。

6.7 真正最复杂的是 ask 分支，因为它要统一四种审批来源

ask 才是这个文件真正的主战场。

如果把它画出来，大概是这样：





这里至少有四条可生效路径：

1. coordinator automated checks
2. swarm worker 转发 leader 审批
3. speculative bash classifier 自动放行
4. 本地交互式权限对话框

这再次证明了一点：

Claude Code 的权限系统不是 modal，而是多来源决策网络。

6.8 coordinator 路径说明“先自动化检查，再惊动用户”是明确策略

`src/hooks/toolPermission/handlers/coordinatorHandler.ts` 很短，但策略非常明确：

1. 先跑 permission hooks
2. 再跑 classifier
3. 两者都无法决策时，才回落到 dialog

这代表一种很成熟的产品思路：

先用机器把明显的情况吞掉，只把真正模糊的情况升级给用户。

而且它对异常的处理也很克制：

自动检查出错，不是让整个流程报错，而是继续回到手动确认。

这类 fallback 很关键，因为权限系统一旦因为某个 classifier 异常直接失效，产品体验会很差。

6.9 swarm worker 路径特别能体现“多 Agent 协作下的权限主权”

`src/hooks/toolPermission/handlers/swarmWorkerHandler.ts` 很有意思。

它表达的是一个重要组织原则：

worker 可以工作，但权限主权仍然属于 leader。

这个 handler 的主线是：

1. 先尝试 classifier 自动通过
2. 不行就 `createPermissionRequest(...)`
3. 通过 mailbox 发给 leader
4. `registerPermissionCallback(...)` 等 leader 回信
5. UI 里显示 `pendingWorkerRequest`

这说明 Claude Code 的 swarm 不是“每个 agent 各自随便跑”，而是：

- 执行可以分布
- 授权可以集中

这对真实团队协作非常重要，因为它避免了 worker 无限度扩大权限边界。

6.10 interactive 路径最精彩的地方，是它在处理“多方同时抢答”

`src/hooks/toolPermission/handlers/interactiveHandler.ts` 是我觉得这组权限代码里最见功力的一块。

它不是简单弹一个对话框，而是在构造一场并发竞争：

- 本地用户可能点允许/拒绝
- bridge 端的 CCR 可能先回
- channel 端的远程消息可能先回
- hooks 可能后台先跑完
- classifier 可能在 grace period 内命中

为了避免多次 resolve，它用了 `createResolveOnce()` 的：

- `claim()`
- `isResolved()`
- `resolve()`

这个细节非常关键。

很多权限系统 bug 就出在这里：

- 弹窗还在，远程先批了
- 用户又点了一次
- 本地和远端重复提交
- 状态错乱

Claude Code 这里明显是针对真实竞争条件做过加固的。

6.11 speculative bash classifier 的 2 秒宽限窗口，是很有产品感的小设计

在 ask 分支里，bash 场景还有一个很值得称赞的小设计：

- 如果存在 speculative classifier check
- 就最多等 2 秒
- 如果高置信命中，就直接自动通过
- 否则再展示 dialog

这本质上是在平衡两种体验：

- 不想太早打断用户
- 也不想为了等 classifier 让界面卡太久

这类“给自动化一点点时间，但不给它无限时间”的设计，非常像成熟产品，而不是纯工程思维。

6.12 abort/cancel 的收口也非常统一

这个文件还有一个容易被忽略的优点：

无论失败、取消、用户中断还是 APIUserAbortError，最后都能收口成同一种权限决策语义。

这里通过：

- `resolveIfAborted()`
- `cancelAndAbort()`
- `catch(error => ...)`
- `finally(clearClassifierChecking)`

把流程统一收进来。

这意味着上层调用方拿到的不会是零散异常，而是一个相对稳定的权限结果。

对大型系统来说，这会显著降低外围复杂度。

6.13 这个模块真正厉害的地方，是把“权限”做成了协作协议

如果只把权限理解成“要不要执行命令”，那这个模块会显得有点重。

但如果把它放进 Claude Code 整个系统里看，你会发现它做的是另一件事：

它让权限决策可以在本地用户、自动规则、远端桥接、swarm leader、classifier 之间流动。

这时候权限就不再只是安全机制，而是一种：

跨角色、跨终端、跨执行主体的协作协议。

这也是为什么我会说 `src/hooks/useCanUseTool.tsx` 虽然体量不大，但架构意义很大。它补上了 Claude Code 从“会调用工具”到“可控地调用工具”之间最关键的一层。

7. `src/services/mcp/client.ts`：外部能力总线的接入工厂

如果说前面几个模块回答的是：

- Claude Code 怎么组织会话
- 怎么组织工具
- 怎么组织权限
- 怎么组织子 agent

那么 `src/services/mcp/client.ts` 回答的就是另一个大问题：

Claude Code 怎么把“外部世界的能力”接进自己的运行时。

这文件非常重，而且不是“发几个 HTTP 请求”那么简单。

它同时承担了至少三层职责：

1. 连接不同类型的 MCP server
2. 把 MCP server 的能力转译成 Claude Code 自己的 Tool / Command / Resource
3. 把 MCP 返回的复杂结果重新加工成模型可消费的内容

所以它不是一个简单 client，而更像：

MCP integration factory + adapter layer + result gateway。

7.1 先说定位：它不是某个 server 的 client，而是一整条 MCP 接入面

从 import 规模就能看出来，这个文件不是面向单一协议 happy path 写的。

它要同时处理：

- `SSEClientTransport`
- `StreamableHTTPClientTransport`
- `StdioClientTransport`
- `WebSocketTransport`
- `SdkControlClientTransport`
- in-process server
- [claude.ai](#) proxy

再加上：

- OAuth
- proxy
- TLS
- session ingress token
- 资源工具
- elicitation
- 大结果落盘

这说明作者对 MCP 的理解不是“多一个插件协议”，而是：

Claude Code 的外部能力总线。

也正因为是总线，它必须处理大量异构情况，而不能只写最干净的那条链路。

7.2 这个文件最值得先注意的一点：它把 transport 多样性当成一等事实

`connectToServer()` 是这章最核心的函数之一。

它不是固定建立某一种连接，而是根据 `serverRef.type` 分流：

- sse
- sse-ide
- ws-ide
- ws
- http
- claudeai-proxy
- stdio
- in-process Chrome MCP
- in-process Computer Use MCP

这说明作者从一开始就接受了一个现实：

MCP 不是一个部署形态，而是一套协议外壳。

真正落地时，server 可能是：

- 本地子进程
- 本地 IDE
- 远程 HTTP 服务
- 远程 SSE 长连接
- WebSocket 服务
- 代理后的 [claude.ai](#) connector

- 当前进程内直接嵌入的 server

很多项目会试图把这些都藏起来，伪装成“统一 client”；Claude Code 反而做得更诚实：它承认这些差异存在，然后在统一入口里显式处理。

7.3 connectToServer() 其实像一个连接工厂，而不是一个普通 connect 函数

我觉得 connectToServer() 最厉害的地方，不只是分 transport，而是它把连接这件事拆成了一个完整生命周期：

- 构造 transport
- 注入认证与 header
- 包装 timeout
- 建立 Client
- 注册 request handler
- 检查 capabilities / version / instructions
- 挂上 error / close handler
- 返回 connected / needs-auth / failed / disabled

所以它输出的不是裸 SDK client，而是：

Claude Code 自己语义下的 MCPServerConnection。

这一步非常关键，因为只要这层抽象立住，后面的上层代码就不必直接感知 SDK 的原始连接细节。

7.4 认证设计说明作者知道“连接失败”里最大的一类其实是 auth

这个文件里和认证有关的细节非常多：

- McpAuthError
- MCP_AUTH_CACHE_TTL_MS
- isMcpAuthCached()
- setMcpAuthCacheEntry()
- handleRemoteAuthFailure()
- createClaudeAiProxyFetch()
- checkAndRefreshOAuthTokenIfNeeded()
- handleOAuth401Error()

这背后的判断我很认同：

MCP 连接失败里最值得单独工程化处理的，不是网络超时，而是认证失效。

作者做了几个非常产品化的处理：

1. 401/needs-auth 不是简单失败，而是独立状态
2. needs-auth 会进入 15 分钟缓存，避免反复探测
3. 有 discovery 但没有 token 的 server 会被跳过，避免空耗
4. [claude.ai](#) proxy 的 bearer token 会在 401 后尝试刷新再重试一次
5. 上层还能对 needs-auth server 生成 `createMcpAuthTool(...)`

这说明 Claude Code 并不是把 auth 当“连接前置条件”，而是把它当成 MCP 生命周期的一部分。

7.5 它对连接稳定性的处理很成熟，明显踩过真实坑

`connectToServer()` 后半段有很多很“脏”但很宝贵的逻辑：

- 连接超时保护
- stderr 捕获
- repeated terminal errors 计数
- Maximum reconnection attempts 特判
- session expired 检测
- close 时清理 memo cache 和 fetch cache

这类代码一眼就能看出不是“按文档写”的，而是被线上问题逼出来的。

特别是这几个点很值得欣赏：

1. SDK 的 `onerror` / `onclose` 不完全可靠，所以作者自己补了关闭和重连触发逻辑
2. 远程 transport 连续错误达到阈值后会主动 `closeTransportAndRejectPending(...)`
3. session 过期通过 HTTP 404 + JSON-RPC `-32001` 双信号识别
4. `client.onclose` 会主动清掉 `connectToServer.cache` 和 `fetch*ForClient.cache`

这说明作者很清楚：

在一个长生命周期 Agent Runtime 里，MCP 最难的不是第一次连上，而是掉线之后不挂死。

7.6 它不是只连 server，还把 MCP 能力转译成 Claude Code 自己的语言

这一块我觉得是这个文件架构价值最高的地方之一。

连接成功之后，Claude Code 不会把原始 MCP schema 直接扔给模型，而是做二次映射：

- `fetchToolsForClient(...)`
- `fetchCommandsForClient(...)`
- `fetchResourcesForClient(...)`

这里分别把 MCP 能力转成：

- Tool

- Command
- Resource

这一步非常关键，因为它说明 Claude Code 的宿主协议高于 MCP 原协议。
MCP 只是外部能力来源，真正给模型消费的还是 Claude Code 自己的能力抽象。

也就是说，Claude Code 并没有让 MCP 反过来定义自己的运行时；
它做的是：

把 MCP 吞进自己的运行时模型里。

7.7 fetchToolsForClient() 特别能体现“协议映射层”这个角色

这段代码不是简单把 `tools/list` 结果拿回来展示，而是做了完整的能力翻译：

- `buildMcpToolName(client.name, tool.name)`
- `mcpInfo`
- `searchHint`
- `alwaysLoad`
- `description() / prompt()`
- `isConcurrencySafe()`
- `isReadOnly()`
- `isDestructive()`
- `isOpenWorld()`
- `toAutoClassifierInput()`
- `checkPermissions()`
- `call()`
- `userFacingName()`

这已经不是“封装 SDK 返回值”，而是在创建一批：

符合 Claude Code Tool 契约的宿主工具对象。

更妙的是，MCP tool 进来之后就自动获得了 Claude Code 的整套宿主能力：

- 权限系统
- 进度事件
- 自动分类器输入
- UI collapse 行为
- user-facing name

这正是平台型架构的力量。

只要外部能力被接成 Tool，它就能复用平台里的很多公共机制。

7.8 getMcpToolsCommandsAndResources() 体现了真正的“总线装配”过程

如果说 `connectToServer()` 是单连接工厂，那么 `getMcpToolsCommandsAndResources()` 就是：

批量接入与装配入口。

它做了很多很成熟的事情：

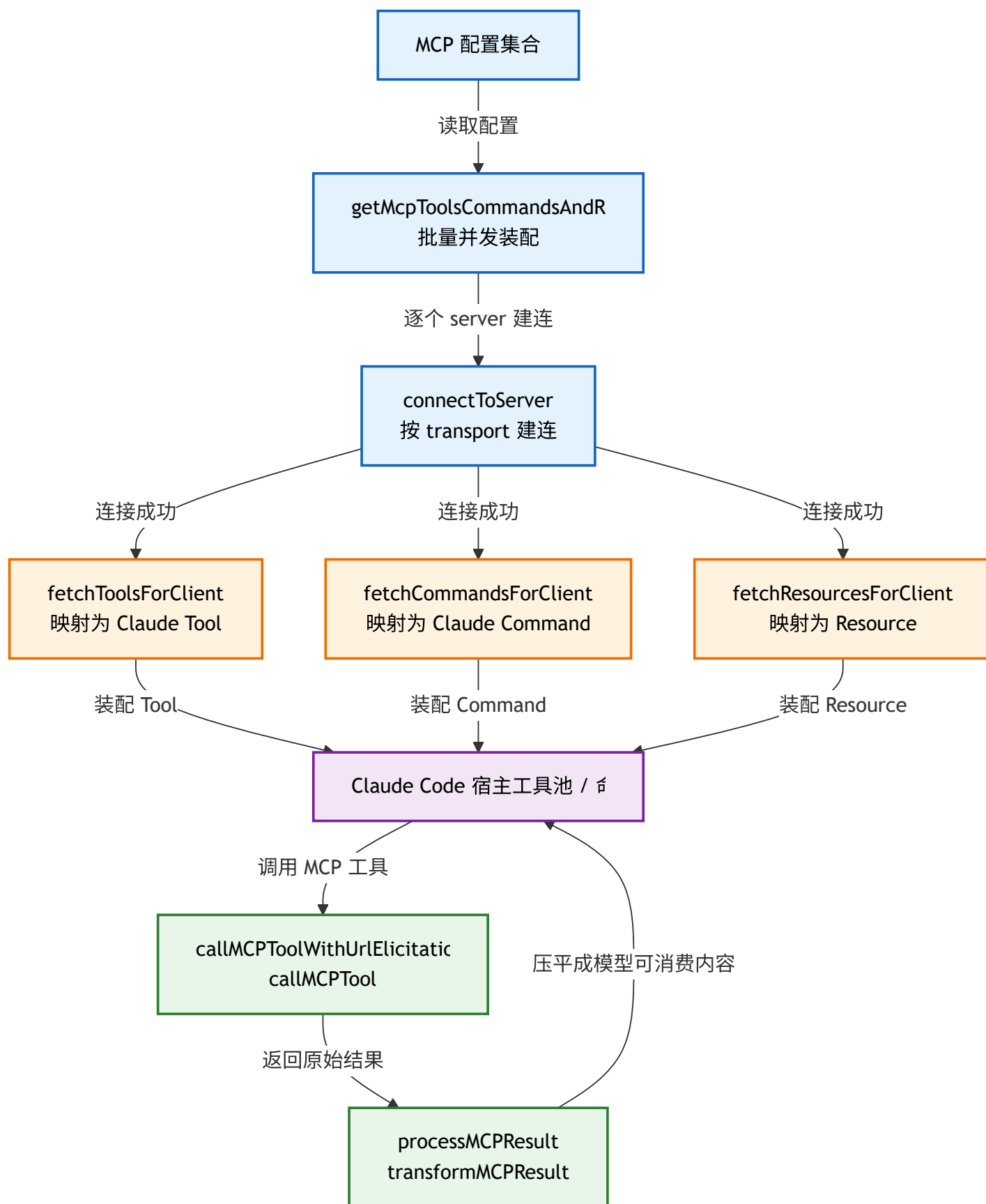
- 先把 disabled server 分出去
- 按本地 server / 远程 server 分组
- 对不同组给不同并发上限
- 对 needs-auth server 直接下发 `createMcpAuthTool(...)`
- 拉 tools / commands / skills / resources
- 只注入一次资源工具 `ListMcpResourcesTool` / `ReadMcpResourceTool`
- 每个 server 完成后通过 `onConnectionAttempt(...)` 增量回流给上层

这说明它不是“等所有连接全结束再统一返回”，而是更像：

一个边连接、边发现、边汇总的能力装配流。

这个设计非常适合 Agent Runtime，因为宿主可以逐步把可用工具开放出来，而不用死等最慢的那个 connector。

7.9 这张图能比较清楚地说明它的三段式职责



这张图里最关键的点是：

它不是“连上 server 然后直接透传”，而是“接入 -> 转译 -> 回收”。

这才是平台总线的写法。

7.10 返回值处理这部分做得非常好，明显不想让 MCP 把上下文炸掉

后半段的：

- `transformResultContent(...)`
- `transformMCPResult(...)`
- `processMCPResult(...)`
- `persistBlobToTextBlock(...)`

非常值得看。

作者面对的显然是一个真实问题：

MCP 返回值可能又大、又杂、又包含图片/音频/二进制/结构化对象，不能直接塞给模型。

所以这里做了很多防爆处理：

- 文本转 text block
- 图片缩放压缩后再转 image block
- 二进制 blob 落盘后只返回路径说明
- structuredContent 会生成 compact schema
- 结果过大时会：
 - 先判断是否需要截断
 - 图片内容走截断而不是 JSON 落盘
 - 普通大结果落盘并返回读取说明

这非常像一个成熟宿主在做的事：

保护上下文窗口，把外部返回值驯化成模型可承受的格式。

7.11 `callMCPToolWithUrlElicitationRetry()` 说明作者把 MCP 的交互式工具也接进来了

很多人理解 MCP tool，会把它想成同步 RPC。

但这里专门有一层：

- `callMCPToolWithUrlElicitationRetry(...)`

它处理的是 MCP 的 `UrlElicitationRequired` 错误，也就是：

tool 调用过程中要求用户去完成一个 URL 交互。

作者做的不是简单报错，而是：

1. 识别 -32042

2. 解析 elicitation 列表
3. 先跑 elicitation hooks
4. 不行再走 `handleElicitation(...)` 或 REPL queue
5. 完成后重试 tool call

这件事很重要，因为它说明 Claude Code 接入 MCP 时，并没有假设外部工具都是“无用户交互”的。

相反，它接受了：

某些外部能力本身就是半交互式协议。

这又一次说明它的架构更像 runtime，而不是调用器。

7.12 `callMCPTool()` 是最后一道“协议脏活收口层”

真正的原始调用在 `callMCPTool()` 里。

它做的事情比你想象得多：

- 长时间工具调用进度日志
- 自己再包一层 timeout
- 将 SDK progress 转成 Claude Code 的 `mcp_progress`
- 处理 `isError: true`
- 特判 401 -> `McpAuthError`
- 特判 session expired / connection closed -> 清缓存并抛 `McpSessionExpiredError`
- 成功后再走 `processMCPResult(...)`

这一层非常像典型的 adapter gateway：

它把 SDK 原始行为和 Claude Code 上层期望之间的缝隙都补上了。

所以如果让我一句话概括它，我会说：

`callMCPTool()` 负责把“不稳定的外部协议行为”翻译成“稳定的宿主错误语义与结果语义”。

7.13 这个模块最值得借鉴的地方

如果让我总结 `src/services/mcp/client.ts` 最值得学的地方，我会给四个关键词：

1. 异构接入
2. 平台转译
3. 结果驯化
4. 连接韧性

异构接入，是它承认 transport 和部署形态的复杂性。

平台转译，是它坚持把外部能力映射成 Claude Code 自己的 Tool/Command/Resource。

结果驯化，是它不让 MCP 原始结果直接冲进上下文。

连接韧性，是它认真处理超时、掉线、401、session 失效和重连。

所以这文件真正厉害的地方，不是“把 MCP 接进来了”，而是：

它把 MCP 从一个外部协议，接成了 Claude Code 自己的内生能力总线。

8. src/state/AppStateStore.ts：运行时状态内核的总台账

前面几章我们已经分别看了：

- 会话怎么跑
- 工具怎么定义
- 权限怎么编排
- MCP 怎么接入
- 子 agent 怎么调度

这时候自然会冒出一个问题：

这么多状态，最后都放在哪里？

Claude Code 的答案，就落在 `src/state/AppStateStore.ts` 这一组文件上：

- `src/state/AppStateStore.ts`
- `src/state/store.ts`
- `src/state/AppState.tsx`
- `src/state/onChangeAppState.ts`
- `src/state/selectors.ts`

其中如果只挑一个“状态层中枢”，我会选 `src/state/AppStateStore.ts`。

因为它做的事情不是写几个字段，而是：

把 Claude Code 这台 Agent Runtime 的“全部显性状态面”定义出来。

8.1 先说判断：这不是普通 UI state，而是整套运行时的共享控制面

很多项目里的 AppState 只是：

- 当前页面
- 当前 tab
- 当前 loading

但 Claude Code 这里完全不是这个层级。

`AppStateStore.ts` 里的 `AppState` 同时覆盖了：

- 会话态
- 权限态
- 任务态
- MCP 态
- 插件态
- remote / bridge 态
- teammate / team 态
- 通知与 elicitation 态
- speculation / promptSuggestion 态
- tmux / browser / computer-use 态

这说明作者对 `AppState` 的理解不是“前端页面状态”，而是：

整个宿主运行时的共享状态总线。

也就是说，Claude Code 的 UI 只是这个状态的一种消费方；
真正被建模的是整套 runtime 本身。

8.2 `AppState` 的体量，本身就是 Claude Code 架构复杂度的地图

如果你顺着 `src/state/AppStateStore.ts` 往下看，会发现这个类型几乎就是 Claude Code 的模块目录展开图。

里面至少能看到这些大的状态簇：

- 基础会话与显示： `verbose`、`statusLineText`、`expandedView`
- 模型与权限： `mainLoopModel`、`toolPermissionContext`
- 远程与 bridge： `remoteSessionUrl`、`replBridge*`
- 任务系统： `tasks`、`foregroundedTaskId`、`viewingAgentTaskId`
- MCP： `mcp.clients`、`mcp.tools`、`mcp.commands`、`mcp.resources`
- 插件： `plugins.enabled`、`plugins.errors`、`plugins.installationStatus`
- agent / team： `agentDefinitions`、`teamContext`、`agentNameRegistry`
- 通知与交互： `notifications`、`elicitation`、`inbox`
- 预测与建议： `promptSuggestion`、`speculation`
- 终端外设与工具态： `tungsten*`、`bagel*`、`computerUseMcpState`

这非常值得注意，因为它说明 Claude Code 的复杂度不是藏在各种 singleton 里，而是：

尽量显式地暴露为统一状态面。

这种设计的好处是，后面你分析系统时，不用到处猜“状态藏哪了”，直接看 `AppState` 就知道这系统到底在管理什么。

8.3 这个文件最重要的价值，不是字段多，而是它把状态边界一次性画清楚了

我觉得 `src/state/AppStateStore.ts` 最值得肯定的一点是：

作者愿意用一个很长的类型，把系统边界公开写出来。

这比很多项目里“状态散落在 hook、context、模块变量、单例对象、React 本地 state”要健康得多。

因为一旦边界清楚，后面很多事情都会变简单：

- 哪些状态可序列化
- 哪些状态是进程内对象
- 哪些状态要跨 UI / runtime 共享
- 哪些字段适合做 selector
- 哪些更新应该触发副作用

Claude Code 在这点上很像工程团队里经验比较足的一类写法：

宁可把状态总表写重一点，也不把状态分散得找不到。

8.4 tasks、mcp、plugins 这些嵌套块，说明它不是扁平 store，而是“领域分仓”

虽然 `AppState` 很大，但它不是一股脑扁平铺开。

作者很明显在按领域分仓：

- tasks
- mcp
- plugins
- notifications
- elicitation
- workerSandboxPermissions
- promptSuggestion
- skillImprovement

这说明他不是没意识到复杂度，而是在做一种很务实的控制：

在一个总 store 下，按运行时子域分组。

这类结构比 Redux 风格的大量 reducer 切片更轻，但又比“所有字段散着放”更可维护。

对一个 CLI + Ink + 多 agent + MCP 的混合系统来说，我觉得这是一个合理折中。

8.5 tasks 被故意排除出 DeepImmutable ，这说明作者对“状态纯度”和“工程现实”有取舍

这个细节很有意思。

AppState 主体包了一层 DeepImmutable<...> ，但 tasks 被单独拿出来，注释里明确说：

- TaskState contains function types

这其实暴露了一个很真实的工程取舍：

作者想让大部分状态都尽量不可变、可预测，但承认任务对象里有些运行时句柄没法完全纯数据化。

这很像成熟工程而不是学院派工程。

真正复杂的运行时里，总会有：

- callback
- abort controller
- cleanup 函数
- 进程句柄

如果硬要把这些都纯数据化，系统会变得很拧巴。

Claude Code 在这里的做法是：

- 大面上保持不可变数据风格
- 局部允许任务态保留运行时对象

这个平衡我觉得是理性的。

8.6 getDefaultAppState() 很重要，因为它不是默认 UI，而是默认宿主

很多人看到 getDefaultAppState() ，会觉得这只是初始化。

但在 Claude Code 里，这个函数的意义更大：

它定义的是“Claude Code 进程刚启动时，整个 runtime 的零点状态”。

而且这份默认态不是随便填的。

它里面有不少有意思的系统级判断：

- teammate 且 plan_mode_required 时，初始 permission mode 设成 plan
- thinkingEnabled 走 shouldEnableThinkingByDefault()
- promptSuggestionEnabled 走 shouldEnablePromptSuggestion()
- toolPermissionContext 从 getEmptyToolPermissionContext() 起步
- mcp 、 plugins 、 notifications 、 elicitation 全都有完整空态

这说明初始化不是“先给个空对象再说”，而是：

把产品默认行为直接写进状态工厂。

这类默认态工厂非常关键，因为它决定了系统在最复杂的 runtime 逻辑启动前，先以什么姿态站稳。

8.7 真正的 store 原语其实非常小，这反而是优点

src/state/store.ts 很短，核心只有：

- getState()
- setState(updater)
- subscribe(listener)

createStore() 的实现也很干净：

- 存 state
- 存 listeners
- setState 用函数式 updater
- Object.is(next, prev) 避免无效通知
- 可选 onChange({ newState, oldState })

这说明 Claude Code 的状态系统没有引入一个笨重状态框架，而是自己保留了最小原语。

这背后的好处很明显：

1. React 外部也能用
2. 非 UI 逻辑也能安全持有 getState/setState
3. 调试时能很清楚知道状态更新机制到底是什么

这也是为什么它特别适合 Claude Code 这种“不是纯 React App”的系统。

因为这套运行时本质上有大量逻辑发生在 React 之外。

8.8 AppState.tsx 说明 React 在这里是消费者，不是状态拥有者

src/state/AppState.tsx 做的事情很清楚：

- 用 createStore(...) 创建 store
- 用 AppStateContext 提供出去
- 用 useSyncExternalStore(...) 做选择性订阅
- 暴露 useAppState()、useSetAppState()、useAppStateStore()

这里最关键的设计取向是：

状态先存在，React 再去订阅它。

而不是：

状态存在于 React 组件树里，其他地方再想办法取。

这对 Claude Code 这种系统太重要了。

因为它有很多更新来自：

- MCP 回调
- 任务系统
- bridge
- server mode
- tool runtime

这些地方很多都不是 React 组件。

所以把 store 放在 React 之下，显然是不对的；Claude Code 的做法是把 React 放在 store 之上。

8.9 useAppState(selector) 的写法很成熟，明显在防大范围重渲染

这里作者没有写一个“拿整个 state”的粗暴 hook，而是明确鼓励 selector 模式：

- useSyncExternalStore(store.subscribe, get, get)
- 注释里提醒不要返回新对象
- 多字段就多次调用 hook

这说明作者意识到了一个现实：

AppState 很大，如果消费方式粗糙，整个 Ink UI 会被状态更新拖垮。

所以他们通过 selector + Object.is 做最小重渲染。

这不是特别花哨，但非常有效，也符合现在 React 官方比较推荐的 external store 用法。

8.10 onChangeAppState() 其实是状态层与副作用层之间的桥

我很喜欢 src/state/onChangeAppState.ts 这个分层。

它的作用非常明确：

不是负责决定状态，而是负责在状态变化后触发必要的外部同步。

里面已经能看到不少典型例子：

- permission mode 变化 -> notifySessionMetadataChanged() / notifyPermissionModeChanged()
- mainLoopModel 变化 -> 更新 settings 与 override
- expandedView / verbose / tungstenPanelVisible -> 持久化到 global config
- settings 变化 -> 清 auth cache、重置 env

这说明作者没有把副作用塞进每个 `setState` call site，而是尽量把“状态变了之后要干什么”收进一个总 diff 点。

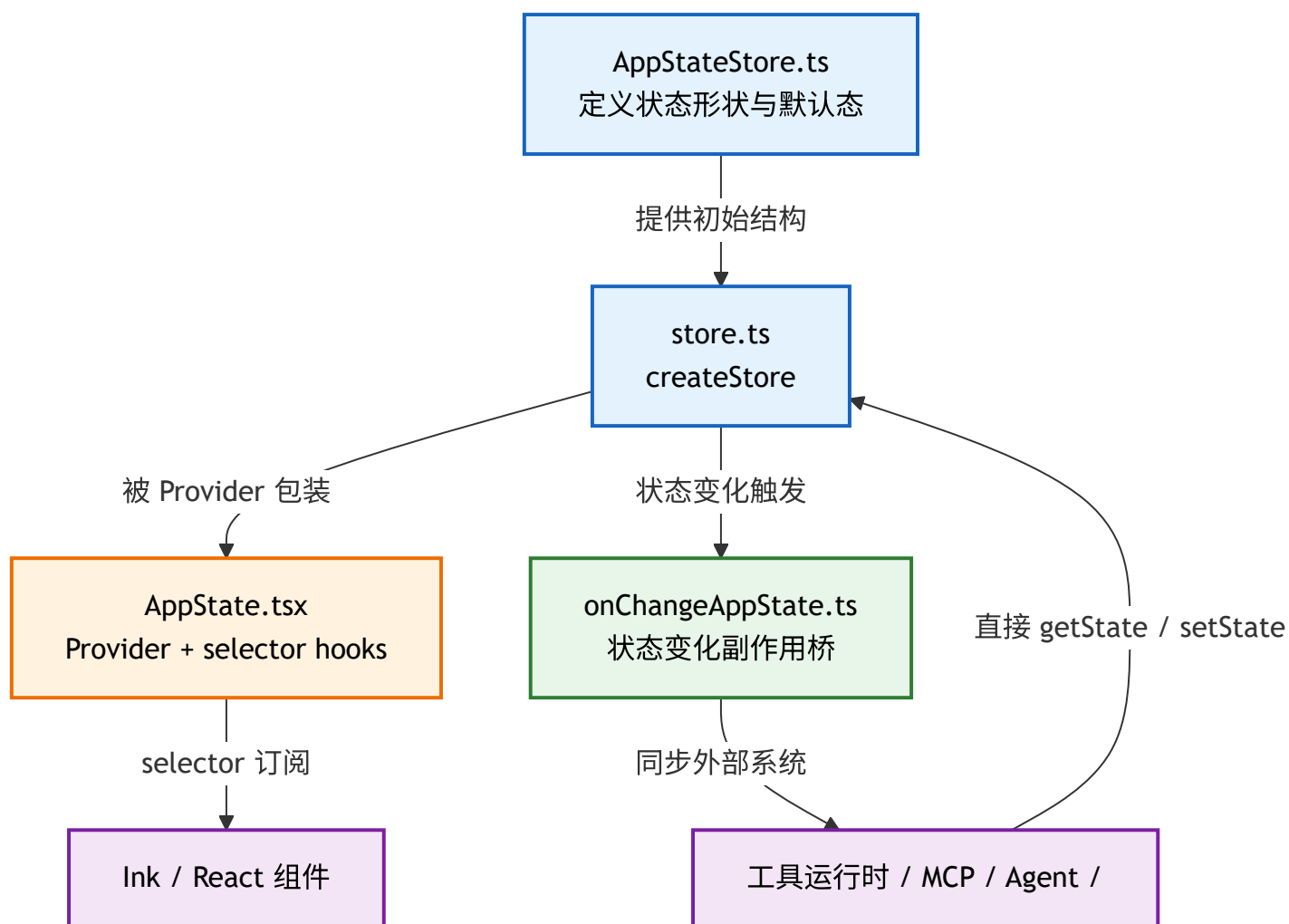
这是非常成熟的设计。

因为一旦没有这个 choke point，后面每个地方都可能：

- 改 state
- 顺手写 config
- 顺手发桥接事件
- 顺手清缓存

最后就很难追。

8.11 这张图能说明它为什么更像运行时状态总线，而不是前端 store



这个图里最关键的一点是：

React 只是右边一个消费方，不是整个状态系统的中心。

这和很多纯前端应用完全不同。

Claude Code 的 store 更像 runtime backbone。

8.12 这套状态层最值得欣赏的地方，是“中心化定义，去中心化消费”

如果让我总结这套状态层设计的优点，我会用一句话概括：

中心化定义，去中心化消费。

中心化定义体现在：

- AppState 一处定义系统状态面
- getDefaultAppState() 一处定义默认宿主
- onChangeAppState() 一处定义关键副作用桥

去中心化消费体现在：

- React 通过 selector 消费
- 非 React 代码直接拿 store
- 任务系统、MCP、bridge 都能经 setState 进入同一总线

这对 Claude Code 这种“宿主 + UI + 工具 + 多 agent + 远程桥接”混合系统来说，我觉得是非常对路的。

8.13 这个模块最值得借鉴的地方

如果让我总结 src/state/AppStateStore.ts 这一组状态层设计，最值得学的是三个关键词：

1. 显式
2. 轻量
3. 统一

显式，是状态边界不藏。

轻量，是底层 store 原语非常小。

统一，是 UI、runtime、副作用都围绕同一份状态总线工作。

所以这组代码真正厉害的地方，不是“字段很多”，而是：

它把 Claude Code 的运行时复杂度收进了一个还能被人理解的状态骨架里。

9. src/bridge/bridgeMain.ts：远端控制与桥接主循环

如果前面几个模块让我们看到的是 Claude Code 的本地 runtime，那么 src/bridge/bridgeMain.ts 看到的就是一面：

Claude Code 怎么把本地进程，变成一个可被远端接入、可被远端恢复、可持续轮询工作的桥接宿主。

这个文件很重，而且它的重不是“代码太长”，而是职责密度非常高。
它至少同时负责：

1. 启动和注册 bridge environment
2. 轮询远端 work queue
3. 根据 work 孵化本地 session 子进程
4. 维持 heartbeat、token refresh、reconnect
5. 管理 worktree 与 session 生命周期
6. 处理 headless 模式与交互模式两套入口

所以我会把它定义为：

Claude Code Remote Control 的主循环调度器。

9.1 先说定位：它不是一层薄桥，而是“远端控制 runtime”

很多系统里的 bridge 只是做两件事：

- 收到请求
- 转发到本地

但 `src/bridge/bridgeMain.ts` 明显不是这种薄层。
它里面有自己的：

- 参数解析
- 模式决策
- environment 注册
- session spawn
- poll/backoff/heartbeat
- shutdown/cleanup
- headless logger

这说明 bridge 在 Claude Code 里不是“补充功能”，而是完整的运行时子系统。

换句话说，Remote Control 不是在 Claude Code 外面包一层壳，
而是把 Claude Code 进程的一部分能力重新组织成一个：

可持续运行的远端会话代理。

9.2 bridgeMain() 负责的是“把一堆配置与前置条件收敛成可运行桥”

bridgeMain(args) 前半段其实很值得看。
它不是立刻去 poll，而是先做了很多前置收敛：

- parseArgs(...)
- multi-session gate 校验
- trust 检查
- auth 检查
- baseUrl / sessionIngressUrl 决定
- spawnMode 决定
- resumeSessionId / continueSession 决定
- environment 注册

这说明作者没有把桥接理解成“只要 token 对了就能开跑”，而是明确认为：

桥接是一种受 workspace trust、auth、spawn mode、resume 语义共同约束的运行模式。

这点非常重要。

因为如果不先把这些条件收敛，后面的 poll loop 再稳，整体体验也会非常脆。

9.3 spawnMode 是这个文件里非常关键的产品抽象

这个文件反复出现 spawnMode，而且不是摆设。
它至少涉及：

- single-session
- same-dir
- worktree

在桥接场景下，这个抽象非常聪明。

因为远端控制不是纯“连不连得上”的问题，而是：

远端发起的新会话，应该怎样落到本地工作区。

这里的三种模式其实对应三种产品哲学：

1. single-session
只恢复一个会话，生命周期强耦合，偏传统 remote attach
2. same-dir
多个会话共享当前目录，追求轻量与即时性
3. worktree
每个会话独立 git worktree，追求并发安全

也就是说，作者在 bridge 层讨论的已经不是传输协议，而是：

远端协作时的工作隔离模型。

这比一般人理解的 bridge 要深很多。

9.4 runBridgeLoop() 才是真正的心脏

整个文件的核心毫无疑问是 `runBridgeLoop(...)`。

如果让我只保留一个函数来代表这套桥接系统，我会留它。

因为它把桥接系统最难的几个问题都放在一起解决了：

- 如何跟踪 active sessions
- 如何轮询工作
- 如何在 at-capacity 时不忙等
- 如何 heartbeat 现有 work item
- 如何更新 token
- 如何在 session done 时 stop work / archive / cleanup
- 如何在 shutdown 时优雅收尾

这已经不是“轮询函数”，而是：

桥接模式下的 session scheduler。

9.5 这套桥接循环不是简单 poll，而是 “poll + heartbeat + backoff + wake” 的组合

我很喜欢 `runBridgeLoop()` 里对空闲和满载状态的处理。

它不是死循环 `pollForWork()`，而是组合了几种节奏控制：

- `pollForWork(...)`
- `heartbeatActiveWorkItems()`
- `createCapacityWake(...)`
- `getPollIntervalConfig()`
- `connBackoff` / `generalBackoff`

这意味着作者已经意识到 bridge 场景下有几种完全不同的运行阶段：

1. 空闲无 work
2. 有 session 但未满载
3. 已满载，需要 heartbeat 但不能继续接单
4. 网络抖动 / token 失效 / reconnect

如果只用一种固定轮询间隔，这些阶段都会处理得很差。

Claude Code 的做法明显更成熟：

让 poll、heartbeat、capacity wake 和 backoff 共同组成桥接循环。

这会让系统既不容易打爆服务端，也不容易在 session 结束后反应迟钝。

9.6 onSessionDone() 体现了非常强的生命周期意识

onSessionDone(...) 这段我很欣赏。

它不是简单记个日志，而是在 session 结束时做一整套生命周期收口：

- 从 activeSessions、sessionWorkIds、sessionIngressTokens 等 Map 删除
- 取消 timeout timer
- 取消 token refresh
- capacityWake.wake()
- 根据状态决定是否 stopWorkWithRetry(...)
- 清理 worktree
- multi-session 下 archiveSession(...)
- single-session 下 controller.abort()

这说明作者很清楚：

session 结束不是一个事件，而是一组系统状态迁移。

这也是为什么我一直说 Claude Code 很像 runtime。

因为这种写法明显是把“会话”当成受控生命周期对象，而不是聊天窗口。

9.7 它对 session 恢复和 token 续租的处理很像生产系统，而不是 demo

bridge 场景里最烦的一类问题，往往不是第一次启动，而是：

- session 还活着，但 token 过期了
- work 已经 ACK 出去了，但客户端掉线了
- 进程重启后怎么接回原来的会话

这个文件在这些问题上做了不少很实战的处理：

- heartbeatActiveWorkItems() 中遇到 401/403 会走 reconnectSession(...)
- createTokenRefreshScheduler(...) 负责在 JWT 到期前刷新
- v2 session 通过 registerWorker(...) 接上
- --continue / --session-id 会尝试 resume 指定 session
- resume 失败时，会区分环境失配与瞬时失败

这背后体现的是一种很强的判断：

Remote Control 不是一次性 attach，而是长期运行中的 lease 管理问题。

这已经是 infra/runtime 级思维了。

9.8 worktree 支持说明 bridge 真的是把“并发远端会话”当成真实需求在做

桥接层里对 worktree 的支持不是装饰，而是系统级能力：

- `spawnMode === 'worktree'`
- `createAgentWorktree(...)`
- `sessionWorktrees`
- `removeAgentWorktree(...)`

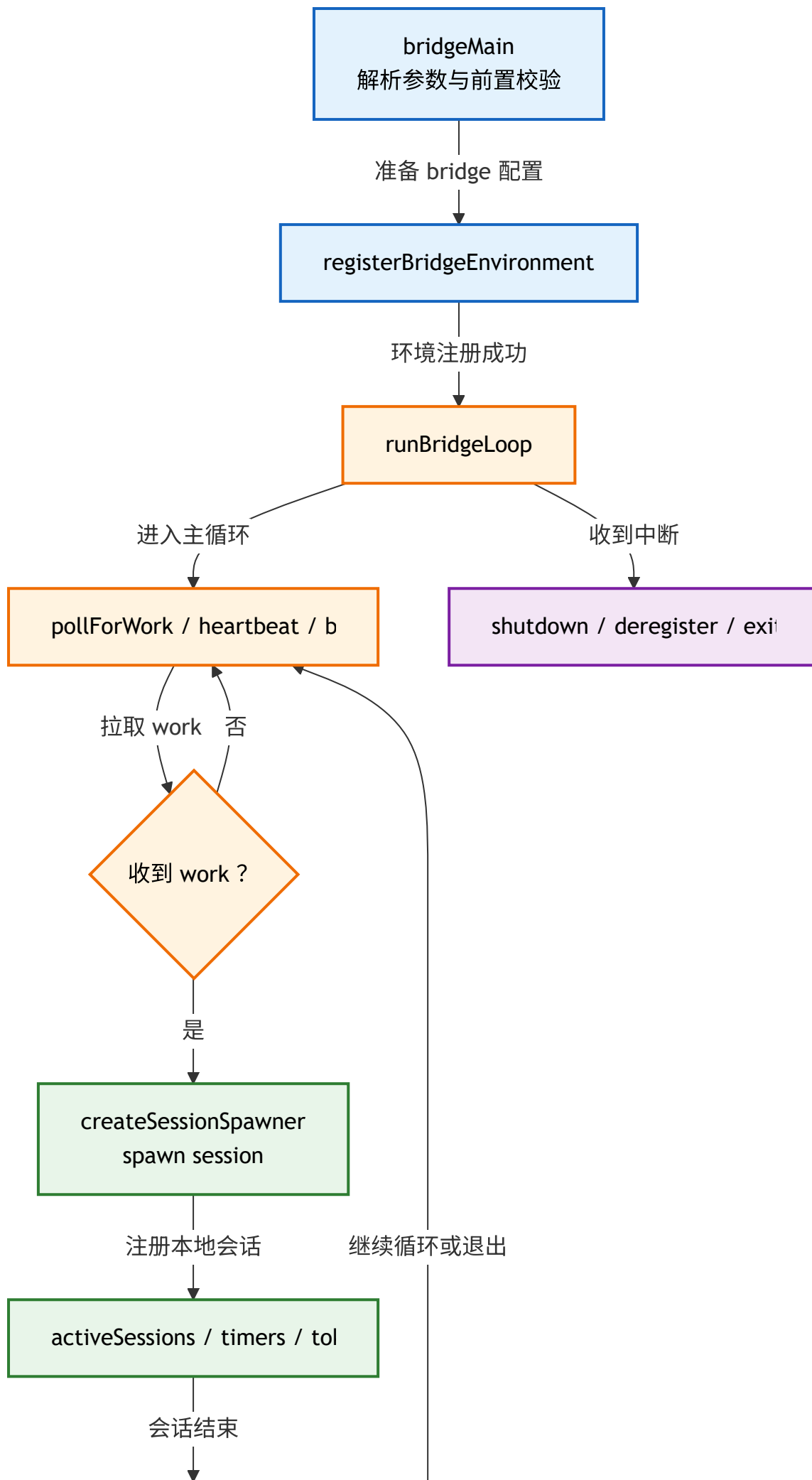
而且作者还特别处理了：

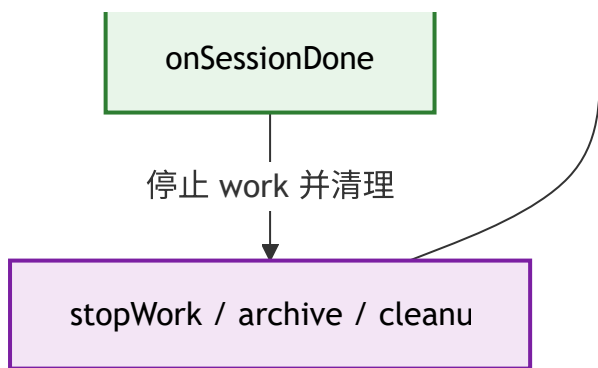
- 预创建的 initial session 不走 worktree，保持旧 UX
- spawn 前就冻结 `spawnModeAtDecision`
- spawn 失败时也会清理 worktree
- shutdown 时统一清理遗留 worktree

这说明远端桥接不是只为了“在网页上看到本地 CLI”，它已经在往“多远端会话并发协作本地仓库”这个方向设计了。

这非常像真正产品化后的 Remote Coding，而不是演示性质的 remote viewer。

9.9 这张图能说明它的核心运行机制





这张图里最重要的一点是：

bridge 的核心不是“转发消息”，而是“托管会话生命周期”。

9.10 bridgeMain() 和 runBridgeHeadless() 双入口说明这层既服务 CLI，也服务守护进程

文件后半段还有个非常重要的入口：

- `runBridgeHeadless(...)`

它不是另写一套桥逻辑，而是：

- 复用 `createBridgeApiClient(...)`
- 复用 `createSessionSpawner(...)`
- 复用 `runBridgeLoop(...)`

只是去掉了：

- `readline` 交互
- `stdin` 键盘切换
- TUI logger
- `process.exit()`

这说明作者在 bridge 层做对了一件事：

把“桥接核心逻辑”和“交互壳层”拆开了。

这样一来：

- CLI 交互模式能跑
- daemon worker 能跑
- supervisor 也能安全托管它

这就让 bridge 不是只服务命令行，而是变成了可嵌入的桥接执行单元。

9.11 这层和权限系统是接得上的，但它自己不试图吞掉权限决策

这点也很成熟。

从 `createSessionSpawner(...)` 的参数能看到：

- `permissionMode`
- `onPermissionRequest(...)`

再结合 `src/bridge/bridgePermissionCallbacks.ts` 和 `src/utils/sessionState.ts`，可以看出来 `bridge` 会把：

- 远端权限请求
- session state / metadata 更新

作为桥的一部分考虑，但它并没有在这里重复发明一套权限系统。

也就是说，它做的是：

会话桥接层负责转运和同步，权限裁决仍然回到统一的权限编排体系。

这个边界划得很好。

如果 `bridge` 自己又内嵌一套权限逻辑，整个系统很快就会分叉。

9.12 优雅关停这部分很见功力

`runBridgeLoop()` 末尾的 `shutdown` 逻辑非常像生产系统：

- `SIGINT / SIGTERM` 触发 `controller abort`
- 给 `child session` 发送 `SIGTERM`
- 超时后 `SIGKILL`
- 等待 `pending cleanups`
- 清理剩余 `worktrees`
- `stopWork(...)`
- `archive / deregister environment`
- 某些 `resumable shutdown` 场景下保留 `pointer`

这明显不是“程序结束前顺手关一下”。

它是在认真处理：

桥接系统停止时，远端环境、本地会话、工作目录、恢复指针分别应该处于什么状态。

这是非常产品化的关停语义。

9.13 这个模块最值得借鉴的地方

如果让我总结 `src/bridge/bridgeMain.ts` 最值得学的点，我会给四个关键词：

1. 主循环
2. 会话托管
3. 恢复语义
4. 关停纪律

主循环，是它把 `poll`、`heartbeat`、`backoff`、`capacity wake` 整成一个调度器。

会话托管，是它把 `session` 当作有完整生命周期的本地子系统。

恢复语义，是它认真对待 `token refresh`、`resume`、`environment mismatch`。

关停纪律，是它把 `stopWork`、`archive`、`worktree cleanup`、`deregister` 分清楚。

所以这文件真正厉害的地方，不是“能远端连上来”，而是：

它把 **Remote Control** 从一个入口功能，做成了一个可持续运行的桥接 **runtime**。

10. `src/server/createDirectConnectSession.ts`：Direct Connect 会话握手入口

10.1 它不是“再写一个请求函数”，而是在补齐 Server Mode 的第一步

如果只看 `src/server/createDirectConnectSession.ts` 这一个文件，你会觉得它很薄：

- 组一个 `POST /sessions`
- 带上 `cwd`
- 可选带 `Authorization`
- 校验返回值
- 组一个内部 `config` 返回

但这正是它容易被低估的地方。

这个文件真正负责的，不是“发请求”本身，而是：

把一个外部 **direct-connect server** 的 **HTTP 建连动作**，转成 **Claude Code** 内部可以继续托管的会话配置。

也就是说，它是一个很薄的握手层，不是一个完整 **runtime**。

runtime 在别处；但没有这一步，**runtime** 根本起不来。

10.2 为什么它重要

Claude Code 到这里已经能看出两套非常不同的运行姿势：

1. 本地进程内直接跑会话
2. 通过 bridge / remote 方式托管远端工作
3. 通过 direct connect 连到一个 server 侧会话

前两种更像“宿主自己掌控全部生命周期”。

第三种则更像“宿主接入一个外部会话承载面”。

而 `src/server/createDirectConnectSession.ts` 正是第三种姿势成立的起点。

它解决的问题是：

- 服务器怎么创建会话
- 当前客户端工作目录怎样传过去
- 权限跳过标志怎样显式表达
- HTTP 返回怎样变成内部 `DirectConnectConfig`
- 后续 WebSocket 连接该连哪一个 `ws_url`

所以它虽然小，但位置非常关键。

它是 **server mode** 的入口握手面。

10.3 它的设计哲学非常“薄协议、厚下游”

这个文件最有意思的地方，是它刻意不做太多事。

它没有：

- 在这里维护长连接
- 在这里处理消息流
- 在这里实现权限请求回调
- 在这里做 session resume
- 在这里解析复杂控制协议

它只做三件核心动作：

1. 把输入压成 server 能理解的最小请求
2. 把响应校验成内部能信任的结构
3. 把错误包装成明确的 `DirectConnectError`

这是一种很成熟的边界切法：

- 握手层只负责“建联成功”
- 连接层负责“会话持续”

- 上层 runtime 再负责“消息编排与产品语义”

这比把所有 direct connect 逻辑塞进一个超级 manager 里更健康。

因为一旦以后 server API 变动，你只需要先修握手面，而不是把消息循环也一起搅乱。

10.4 cwd 与 dangerously_skip_permissions 暴露了它在传递“会话初始化语义”

请求体非常短，但很说明问题：

- `cwd`
- 可选 `dangerously_skip_permissions: true`

这意味着 direct connect 建的不是一个抽象 socket，而是一个：

- 带工作目录
- 带权限模式
- 准备运行 Claude 会话

的初始化请求。

这里很值得注意的一点是：

`dangerously_skip_permissions` 没有被柔化成一个模糊配置名，而是直接把危险性写进字段名里。

这和 Claude Code 前面权限系统的风格是一致的：

- 危险能力不靠注释提醒
- 危险能力直接进 API 命名
- 使用者必须显式承担语义成本

这种命名方式很“产品化”，也很“安全工程化”。

10.5 它真正做的，是把外部响应折叠成内部 `DirectConnectConfig`

看它的返回值就很清楚了：

- `config.serverUrl`
- `config.sessionId`
- `config.wsUrl`
- `config.authToken`
- `workDir`

这里最重要的不是字段本身，而是一次 **语义折叠**：

- server 返回的是一个建会话的 HTTP 响应

- Claude Code 内部真正需要的是一个能继续接管的连接配置

也就是说，外部协议世界里是：

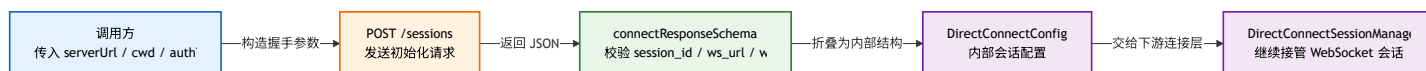
- session_id
- ws_url
- work_dir

内部 runtime 世界里则是：

- DirectConnectConfig
- 后续 WebSocket
- 会话消息回调
- 权限请求回调

这一步折叠做完之后，上层代码就不必继续知道 /sessions 的 HTTP 细节了。

从架构角度看，这就是一个很标准的 anti-corruption layer。



10.6 connectResponseSchema() 说明作者不相信“服务端一定返回对”

src/server/types.ts 里这段 schema 很短：

- session_id: string
- ws_url: string
- work_dir?: string

但它的意义很大。

很多同类系统做到这一步时，会直接：

- await resp.json()
- 然后手写字段访问
- 失败就让异常自然冒泡

Claude Code 没这么做，而是坚持经过 connectResponseSchema().safeParse(...)。

这说明作者对 direct connect 的定位很清楚：

- 它不是“内部函数互调”
- 它是“跨进程 / 跨宿主 / 可能跨版本”的协议边界

只要是协议边界，就必须校验。

否则 server 版本稍微漂一点，上层 runtime 就会带着半坏状态继续运行。

10.7 DirectConnectError 说明这里在主动建构错误语义，而不是放任底层异常泄漏

这个文件另一个我很喜欢的点，是它专门定义了 `DirectConnectError`。

这看起来只是一个薄包装，但价值很实际：

1. 上层可以明确知道这类错误属于 `direct connect` 握手失败
2. 网络失败、HTTP 失败、响应解析失败都能被统一归类
3. UI / CLI / server mode 后续要做差异化提示时有了稳定类型

它没有把：

- `fetch` 错误
- `resp.ok` 错误
- `schema parse` 错误

原样甩给上游，而是统一收束成一个产品语义错误。

这和很多“工程上能跑就行”的实现差别很大。

Claude Code 明显更在意：**上层最终拿到的到底是不是一个可叙述、可处理、可呈现的错误。**

10.8 跟 `src/server/directConnectManager.ts` 连起来看，层次就完整了

单看 `src/server/createDirectConnectSession.ts` 会觉得它很薄；
但和 `src/server/directConnectManager.ts` 一拼，层次就很清楚：

- `createDirectConnectSession.ts` 负责建 session
- `directConnectManager.ts` 负责维持 session

后者真正处理的是：

- 建 `WebSocket`
- 收 `SDKMessage`
- 过滤 `keep_alive`
- 识别 `control_request`
- 处理 `can_use_tool` 权限请求
- 回发 `control_response`
- 支持 `interrupt`
- 管理连接状态与断开

所以 `direct connect` 不是一个文件，而是一个分层模型：

1. HTTP 握手层

2. WebSocket 会话层
3. 上层 runtime / REPL / headless runner 集成层

这才是这组模块真正值得学的地方。

不是“代码多高级”，而是它把一个跨进程会话问题拆成了正确的三个边界。

10.9 directConnectManager.ts 暴露了 Claude Code 对“远端会话”采用的是流式控制协议，而不是 REST 轮询

src/server/directConnectManager.ts 里，消息接入是 WebSocket，且是逐行 JSON 解析：

- message 事件
- data.split('\n')
- jsonParse(line)
- 识别 StdoutMessage

然后再把不同类型分流：

- control_request
- control_response
- keep_alive
- streamlined_text
- streamlined_tool_use_summary
- system.post_turn_summary
- 其他真正需要上送给宿主的 SDKMessage

这说明 direct connect 的本质不是“远端 HTTP API”，而是：

服务端托管 Claude 会话，客户端通过结构化流协议接管交互。

也就是说，createDirectConnectSession.ts 只是拿到这条流的入场券。

10.10 这套设计的优点与代价

优点很明显：

1. 握手层足够薄，便于维护
2. 错误语义清楚，便于产品层提示
3. schema 校验严谨，适合协议边界
4. 与 directConnectManager.ts 的职责分层很干净
5. 给 REPL / headless runner 提供了统一的 direct connect 入口

代价也有：

1. 逻辑分散在多个文件里，第一次读不容易一眼看全

2. `createDirectConnectSession.ts` 太薄，单文件可见价值容易被低估
3. direct connect 协议的真实复杂度被“藏”到了 manager 和上层 runtime 中

但总体上，我会认为这不是缺点，而是成熟架构常见的观感：

真正重要的边界，往往长得都不显眼。

10.11 这个模块最值得借鉴的地方

如果让我总结 `src/server/createDirectConnectSession.ts` 最值得学的点，我会给四个关键词：

1. 握手边界
2. 协议折叠
3. schema 校验
4. 错误语义

它的价值不在“逻辑多”，而在“位置准”。

它非常准确地站在 server API 和本地 runtime 之间，把：

- 外部响应
- 内部配置
- 失败语义

收成一个干净的入口。

这就是为什么我会说，这文件虽然薄，但它不是边角料。

它是 Claude Code 在 `server / direct connect` 这条支线上的 **会话握手层**。

11. `src/utils/plugins/loadPluginCommands.ts`：插件内容接入宿主命令协议的总适配层

11.1 先说判断：它不是“扫插件目录”，而是在做插件内容协议转换

如果你把 `src/utils/plugins/loadPluginCommands.ts` 当成一个普通 loader，就会低估它。

它表面上在做这些事：

- 读取插件目录
- 找 `.md`
- 读 frontmatter
- 生成 slash command

但真正发生的事情是：

它把插件作者写的 Markdown 内容，转换成 Claude Code 宿主内部统一可执行的 Command 协议对象。

这意味着它不是文件系统工具，而是一个协议适配层。

插件世界里的材料很多：

- `commands/` 目录
- `skills/` 目录
- `plugin.json` 里的 `commands`
- `manifest` 里的 `metadata override`
- 内联 `content`
- 用户配置项
- `${CLAUDE_PLUGIN_ROOT}`、`${CLAUDE_SKILL_DIR}` 之类变量
- `prompt` 里的 `shell` 执行

而这个文件做的，就是把这些异构输入压成 Claude Code 自己那套统一命令模型。

11.2 为什么它重要

插件体系真正要回答的问题不是“能不能装插件”，而是：

插件写出来的东西，最后怎样以宿主能理解、能授权、能展示、能执行的方式挂进去。

`src/utils/plugins/loadPluginCommands.ts` 就是这个答案。

如果没有这层，插件命令会很快退化成：

- 一堆特殊格式
- 一堆分支加载器
- 一堆插件专属执行逻辑

最后宿主里的 `slash command` 体系会被插件体系撕裂。

而 Claude Code 没走这条路，它坚持把插件命令折叠回统一 `Command`。

11.3 这文件最厉害的地方，是命令与技能在这里被部分统一了

虽然文件名叫 `loadPluginCommands.ts`，但它其实同时负责两件事：

- `getPluginCommands()`
- `getPluginSkills()`

也就是说，插件里的 `command` 和 `skill`，在这里并不是两套完全不同的协议，而是：

- 来源不同

- 命名不同
- `loadedFrom` / `progressMessage` / `base directory` 语义略有不同

但最终都被转成同一种 `Command`。

这就是 Claude Code 很成熟的地方：

- 用户体验层可以区分“命令”和“技能”
- 运行时协议层尽量复用同一种抽象

11.4 `pluginLoader.ts` 负责“发现插件内容”，而这个文件负责“把内容变成命令对象”

从 `src/utils/plugins/pluginLoader.ts` 可以看到，`loader` 负责把插件的内容路径先整理出来：

- 自动探测 `commands/`
- 自动探测 `skills/`
- 读取 `manifest` 里的 `commands`
- 解析 object mapping 形式的 `commandsMetadata`
- 把 `source` 路径转成绝对路径
- 过滤掉不存在的路径并记错误

而 `src/utils/plugins/loadPluginCommands.ts` 解决的是：

- 这些路径里的 Markdown 如何解释
- 命令名怎么生成
- `skill` 目录如何折叠
- `frontmatter` 怎么转成 `Command`
- `prompt` 运行前要做哪些变量替换和安全收口

这是非常标准的两段式设计：

1. 资源发现
2. 协议适配

11.5 它支持的输入形态很多，说明作者真把插件当成产品生态在做

从 `schema` 和实现可以看出，插件命令并不只支持“丢几个 Markdown 文件进去”这么简单。

它至少支持这些来源：

1. 插件默认 `commands/` 目录
2. `manifest` 中声明的额外 `command` 路径
3. 单个 `.md` 文件
4. 目录下递归扫描到的多个 `.md`

5. SKILL.md 技能目录
6. manifest object mapping 形式的命令定义
7. manifest 里直接内联的 content

这说明作者心里预设的插件作者类型并不单一：

- 有的人喜欢目录约定
- 有的人喜欢 manifest 显式声明
- 有的人想做单文件命令
- 有的人想做技能目录
- 有的人甚至希望把内容直接内联在 manifest

11.6 collectMarkdownFiles() + transformPluginSkillFiles() 很像内容发现层

collectMarkdownFiles() 做的是：

- 用 walkPluginMarkdown() 递归扫 Markdown
- 读文件
- 解析 frontmatter
- 生成 PluginMarkdownFile

而 transformPluginSkillFiles() 会先按目录分组，然后判断：

- 如果一个目录里存在 SKILL.md
- 那这个目录只保留 skill 文件
- 其他 Markdown 文件不再同时作为普通 command 暴露

这其实是在收敛歧义。

一旦一个目录被识别为 skill 目录，它就不应该再同时被当成：

- 一个技能目录
- 又是一堆普通命令文件

11.7 命名策略做得很好：它把插件命名空间显式带进 slash command

getCommandNameFromFile() 对 skill 和普通文件分别处理：

- 普通文件：取文件名去掉 .md
- skill：取 SKILL.md 所在目录名

同时还会把相对路径折成 namespace，最后生成：

- pluginName:commandName

- `pluginName:namespace:commandName`

这种命名方式的价值不只是“避免重名”，而是：

1. 插件来源被显式编码进名字
2. 多级目录结构能自然映射成命令空间
3. slash command 面板里一眼能看出归属
4. 与内建命令、动态技能、MCP 命令能长期共存

11.8 `createPluginCommand()` 才是真正的核心：插件内容在这里被折叠成 Command

这个函数是全文件真正的核心。

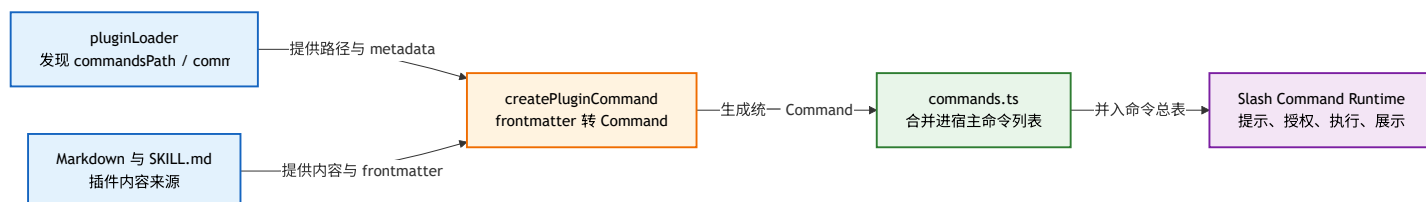
它把一份插件 Markdown，折叠成宿主命令协议时，处理了很多层语义：

- `description`
- `allowedTools`
- `argument-hint`
- `arguments`
- `when_to_use`
- `version`
- `name`
- `model`
- `effort`
- `disable-model-invocation`
- `user-invocable`
- `shell`

最后返回的是 `src/types/command.ts` 里的 `PromptCommand`。

这件事很关键。

因为它意味着插件命令不是一个“外挂执行器”，而是直接变成 Claude Code 的第一等命令对象。



11.9 frontmatter 处理非常成熟：插件 Markdown 更像轻量命令规格

Claude Code 在这里不是简单解析 YAML，而是系统化处理：

- `coerceDescriptionToString()`
- `extractDescriptionFromMarkdown()`
- `parseSlashCommandToolsFromFrontmatter()`
- `parseArgumentNames()`
- `parseUserSpecifiedModel()`
- `parseEffortValue()`
- `parseBooleanFrontmatter()`
- `parseShellFrontmatter()`

这说明插件 Markdown 在 Claude Code 里不是“纯文本 prompt 文件”，而更像：

一份轻量声明式命令规格。

11.10 变量替换链路很强，说明作者把插件内容当成“参数化模板”而不是死文本

`getPromptForCommand()` 里最值得看的，是那条多层替换链：

1. 参数替换 `substituteArguments(...)`
2. 插件变量替换 `substitutePluginVariables(...)`
3. 用户配置替换 `substituteUserConfigInContent(...)`
4. `CLAUDE_SKILL_DIR`
5. `CLAUDE_SESSION_ID`
6. shell 命令执行 `executeShellCommandsInPrompt(...)`

这说明插件内容并不是写完就原样送给模型，而是一个运行时模板。

11.11 `executeShellCommandsInPrompt()` 特别关键：插件 prompt 也能触发受控的环境求值

这一步说明插件 `command` / `skill` 不是静态 prompt，而是：

- 可以在 prompt 生成阶段执行 shell
- 可以把执行结果注入最终 prompt
- 仍然通过 Claude Code 自己的工具权限上下文来约束

更精彩的是这里对 `allowedTools` 的处理。

它会包装一个新的 `getAppState()`，把当前命令允许的工具集合注入

`toolPermissionContext.alwaysAllowRules.command`，也就是说插件 prompt 里的 shell 执行并不是脱离权限系统的后门。

11.12 commandsMetadata 和 inline content 支持，说明它已经开始支持“manifest 驱动的命令定义”

从 `src/utils/plugins/schemas.ts` 和 `src/utils/plugins/pluginLoader.ts` 可以看到，插件命令不仅能靠文件系统约定，还能靠 manifest object mapping 驱动：

- `source`
- `content`
- `description`
- `argumentHint`
- `model`
- `allowedTools`

然后在这个文件里再把这些 metadata override 压回 frontmatter 语义。

这其实很像一种双层模型：

1. 内容层：Markdown / inline content
2. 发布层：manifest metadata

11.13 它对缓存、并发和重复路径的处理，说明作者预设了插件生态会变大

`getPluginCommands()` 和 `getPluginSkills()` 都用了 `memoize(...)`，并且有：

- `clearPluginCommandCache()`
- `clearPluginSkillsCache()`

更重要的是，它不是串行地一个插件一个插件慢慢扫，而是：

- `enabled plugins` 并行处理
- 每个插件内部的多个 `path` 也并行处理
- 用 `loadedPaths` 防重
- 配合 `isDuplicatePath()` 收敛重复来源

11.14 `--bare` 与 “only enabled plugins” 的判断说明插件不是宿主硬依赖，而是可控增量层

`getPluginCommands()` 开头有两个很重要的约束：

1. `--bare` 时，默认跳过 `marketplace` 自动加载
2. 只从 `enabled plugins` 加载

这说明插件在 Claude Code 的定位不是基础生命线，而是：

- 默认可接入
- 但可以整体降级
- 可以按启用状态裁剪
- 可以通过 session 级 `--plugin-dir` 注入

11.15 从 `src/commands.ts` 回看，这层最终是被并进命令总表的

在 `src/commands.ts` 里可以看到：

- 它导入 `getPluginCommands()`
- 也导入 `getPluginSkills()`
- 再和内建命令、bundled skills、dynamic skills 合并

而且文里明确写了：

dynamic skills 插在 plugin skills 之后、built-in commands 之前。

这意味着插件命令不是边角挂件，而是真正进入 Claude Code 主命令表排序体系的一部分。

11.16 这文件的优点与代价

优点很明显：

1. 把插件 command / skill 尽量统一进同一种 Command 协议
2. 支持多种来源形态，生态适配能力强
3. frontmatter 解析和变量替换链非常成熟
4. 权限语义能继续传导到 prompt shell 执行
5. 并发、缓存、防重都做了，适合规模增长

代价也同样明显：

1. 文件职责很重，第一次读会觉得杂
2. command 与 skill 同居一文件，理解成本较高
3. manifest metadata、目录约定、inline content 三套入口叠在一起，抽象密度很高
4. 真正的上下文还要结合 `pluginLoader.ts`、`commands.ts`、`command.ts` 一起看

11.17 这个模块最值得借鉴的地方

如果让我总结 `src/utils/plugins/loadPluginCommands.ts` 最值得学的点，我会给五个关键词：

1. 协议折叠
2. 命名空间治理
3. 模板实例化
4. 权限延续

5. 生态兼容

它最厉害的地方不是“支持插件”，而是：

它没有让插件发明一套自己的命令系统，而是强行把插件内容折回 Claude Code 现有的命令协议。

12. src/skills/loadSkillsDir.ts：技能系统真正成立的总装层

12.1 先说判断：这不是“读 skills 目录”，而是 Claude Code 的技能运行时装配器

如果说 `src/utils/plugins/loadPluginCommands.ts` 解决的是“插件内容如何变成命令”，那么 `src/skills/loadSkillsDir.ts` 解决的就是：

Claude Code 到底怎样把技能当成一等运行时能力接进来。

这个文件干的事远比“扫描目录”重：

- 解析 skill frontmatter
- 生成统一的 Command
- 管理 user / project / policy / legacy / bundled / mcp 等多来源技能
- 去重
- 条件激活
- 动态发现
- 发出变更信号

所以它其实是技能系统的总装层，而不是一个普通 loader。

12.2 最关键的设计：技能最终不是特殊对象，而是被压回 Command

这个文件最重要的函数之一是 `createSkillCommand(...)`。

它最后返回的仍然是 `src/types/command.ts` 里的 `Command`，只是额外带上了：

- hooks
- skillRoot
- context
- agent
- paths
- loadedFrom

这说明 Claude Code 对技能的理解非常成熟：

- 产品概念上，skill 是 skill
- 运行时协议上，skill 尽量还是 command

这带来的好处非常大。

因为只要技能落回 Command ，后面：

- slash command 面板
- model invocation
- prompt 注入
- 权限继承
- cache 失效
- 动态发现

都不用重新发明一套。

12.3 parseSkillFrontmatterFields() 暴露了 Claude Code 对技能的理解不是“提示词片段”，而是“轻量能力声明”

这个函数会系统化解析：

- description
- allowed-tools
- argument-hint
- arguments
- when_to_use
- version
- model
- disable-model-invocation
- user-invocable
- hooks
- context
- agent
- effort
- shell

这很能说明问题。

在 Claude Code 里，skill 不是一段孤立 prompt，而更像一份：

轻量声明式能力规格。

这也是为什么技能系统能跟命令系统、插件系统、MCP skill 系统一起并存。

因为它们最后都落在一套足够通用的运行时抽象上。

12.4 它处理的不只是 `/skills/`，还在兜住历史兼容

`src/skills/loadSkillsDir.ts` 里有一条很重要的双轨逻辑：

1. 新式 `/skills/` 目录
2. 旧式 `/commands/` 目录里的技能写法

也就是：

- `loadSkillsFromSkillsDir(...)`
- `loadSkillsFromCommandsDir(...)`

前者只认 `skill-name/SKILL.md`。

后者则兼容：

- `SKILL.md` 目录形式
- 单个 `.md` 文件形式

这说明作者没有把技能系统做成一次性重构，而是认真对待了历史包袱。

从产品演进角度看，这是非常必要的，因为社区内容和用户本地资产都可能建立在旧约定上。

12.5 多来源技能装配，才是这文件真正的重量

`getSkillDirCommands(cwd)` 才是这文件的中心。

它同时装配这些来源：

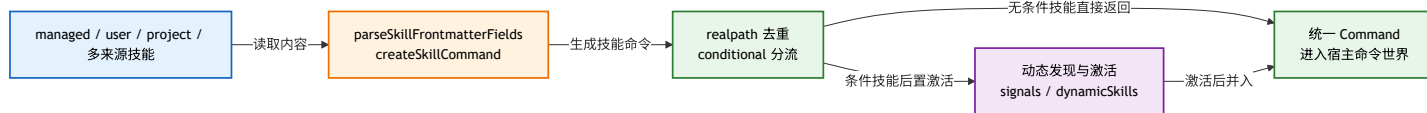
- managed skills
- user skills
- project skills
- `--add-dir` 带来的 additional dirs
- legacy commands-as-skills

然后再去做：

- 去重
- 条件技能分离
- cache
- 激活跟踪

这已经不是“读目录”了，而是在做一张技能来源总表。

也就是说，Claude Code 不是把 skill 作为某个单独目录的附属功能，而是把它当作跨来源的正式能力层。



12.6 去重方式非常讲究：它不是比路径字符串，而是比文件身份

这里我很喜欢的一个点是 `getFileIdentity(filePath)`。

它不是简单拿字符串路径比重，而是走 `realpath()`：

- 处理 symlink
- 处理重叠父目录
- 尽量避免不同路径指向同一文件导致的重复加载

这说明作者遇到过真实世界里的复杂文件系统环境。

这种去重不是“代码洁癖”，而是插件/技能生态一旦规模起来后，必须认真处理的现实问题。

12.7 条件技能是这个文件最有产品感的设计之一

`paths frontmatter` 的支持非常重要。

这意味着某些技能并不是启动时一股脑全部暴露，而是先进入 `conditionalSkills`，只有当文件路径匹配时才激活：

- `conditionalSkills`
- `activatedConditionalSkillNames`
- `activateConditionalSkillsForPaths(...)`

这背后的设计很聪明：

- 减少默认可见技能数量
- 避免技能列表膨胀
- 把 skill 与文件上下文绑定
- 让模型在“碰到相关文件后”才看到相关技能

这就是典型的 Agent 产品思路：

不是能力越多越好，而是要在正确时机显露正确能力。

12.8 动态技能发现说明技能系统不是启动期静态配置，而是会话期可增量演化

`discoverSkillDirsForPaths(...)` 和 `addSkillDirectories(...)` 很关键。

它们说明 Claude Code 的技能系统不是：

- 启动时扫一遍
- 之后固定不变

而是：

- 随着会话中文件操作发生
- 向上遍历文件路径
- 发现新的 `.claude/skills`
- 动态加载
- 发信号通知外层清 cache

这说明技能系统已经和真实工作流绑在一起了。

技能不是一个“安装后静态列表”，而是一个会随着工作上下文展开的能力面。

12.9 它和权限系统是接得上的，但方式很克制

`createSkillCommand(...).getPromptForCommand(...)` 里仍然会调用 `executeShellCommandsInPrompt(...)`，并且和前面插件命令一样，把 `allowedTools` 注入权限上下文。

但它也额外做了一件非常成熟的事：

MCP skills 禁止执行 markdown body 里的 shell。

这一句特别能说明作者对“本地技能”和“远端技能”的信任边界是清醒的。

不是所有 skill 都被一视同仁地当成本地可信内容。

12.10 这个模块最值得借鉴的地方

如果让我总结 `src/skills/loadSkillsDir.ts` 最值得学的点，我会给五个关键词：

1. 技能折叠
2. 多来源装配
3. 条件激活
4. 动态发现
5. 信任分层

它真正厉害的地方，是把 skill 从“提示词目录”做成了：

可声明、可发现、可去重、可动态激活、可回收到统一命令协议中的正式运行时能力。

13. src/server/directConnectManager.ts : Direct Connect 的流式会话控制器

13.1 如果 createDirectConnectSession.ts 是握手层，这里就是持续运行层

前面拆 src/server/createDirectConnectSession.ts 时，我已经说过，它负责：

- 建 session
- 取 ws_url
- 折叠成 DirectConnectConfig

而 src/server/directConnectManager.ts 真正负责的是：

把这个 config 变成一条持续运行的、可收发消息的、带控制语义的会话流。

所以这两个文件的关系非常清楚：

- createDirectConnectSession.ts 负责入场
- directConnectManager.ts 负责演出

13.2 它不是普通 WebSocket wrapper，而是结构化流协议的本地端点

这个文件看起来像一个薄 WebSocket 封装，但其实不是。

它里面已经清楚地表达出几个协议层事实：

- 收到的数据不是任意消息，而是逐行 JSON
- 上游说的是 Claude SDK / StructuredIO 的消息格式
- 消息里混合了内容流和控制流
- 本地端必须理解 permission request / interrupt / keep_alive 这类控制语义

也就是说，它不是一个 transport wrapper，而是：

Direct Connect 流式控制协议在本地宿主里的会话端点。

13.3 最值得注意的一点：它显式区分“内容消息”和“控制消息”

message 处理里有一段很关键：

- 先 split('\n')
- 再 jsonParse(line)
- 再判断 StdoutMessage

- 然后按 `parsed.type` 分流

这里最重要的不是实现细节，而是协议建模：

- `control_request`
- `control_response`
- `keep_alive`
- `control_cancel_request`
- `streamlined_text`
- `streamlined_tool_use_summary`
- `system.post_turn_summary`
- 其他真正上送给宿主 UI / runtime 的 `SDKMessage`

这说明 `direct connect` 流里并不只有“助手输出”，而是混合了一套完整的控制面。

13.4 `can_use_tool` 权限请求接入点，说明 `direct connect` 不是弱客户端

最关键的控制消息是：

- `control_request`
- `subtype === 'can_use_tool'`

一旦遇到这个 `subtype`，它就把请求交给 `onPermissionRequest(...)`。
这意味着 `direct connect` 客户端并不是一个被动看输出的终端，而是：

- 会收到权限审批请求
- 需要本地决策或转交决策
- 再把结果回写到流里

也就是说，远端托管并没有抹平 Claude Code 的权限系统。
权限主权仍然保留在宿主这一侧。

13.5 它对未知控制消息的处理很像生产系统，而不是 demo

我很喜欢这段逻辑：

- 如果收到不认识的 `control_request.subtype`
- 不只是打日志
- 还会 `sendErrorResponse(...)`

这点非常重要。

因为流协议里如果上游在等一个响应，而本地端只是无声忽略，服务端就会挂在那里。

所以这个设计的成熟点在于：

它知道协议错误不只是“报个错”，而是要把错误也按协议回传。

13.6

`sendMessage()`、`respondToPermissionRequest()`、`sendInterrupt()` 共同构成了最小可用控制面

这三个函数把这条 `direct connect` 会话最核心的三类出站动作定义清楚了：

1. 发送用户消息
2. 回应权限请求
3. 发送中断信号

也就是说，这个 `manager` 没试图包揽所有运行时逻辑，但它把 `direct connect` 会话最必要的交互原语都收齐了。

这是很克制的设计。

只暴露真正要给上层用的最小集合。

13.7 它其实就是把 REPL / headless runner 和远端 session 之间插了一层协议翻译器

从 `DirectConnectCallbacks` 来看，这层真正暴露给上游的是：

- `onMessage`
- `onPermissionRequest`
- `onConnected`
- `onDisconnected`
- `onError`

也就是说，上层代码不需要自己理解 `WebSocket` 细节和控制协议细节，只需要响应这些回调。

这就把 `direct connect` 的复杂度成功压在了边界层里。

这是一个很典型的 `runtime adapter` 写法。

13.8 这个模块最值得借鉴的地方

如果让我总结 `src/server/directConnectManager.ts` 最值得学的点，我会给四个关键词：

1. 流式协议分流
2. 控制面保留
3. 错误按协议返回
4. 最小交互原语

它的价值在于：

不是“把 ws 连上”，而是把 Claude Code 的：

- 消息流
- 权限流
- 中断流

统一收成一个可持续运行的远端会话控制器。

14. `src/utils/plugins/pluginLoader.ts`：插件生态的资源发现、校验与合并工厂

14.1 这是 Claude Code 插件系统里最像“供应链后端”的文件

`src/utils/plugins/pluginLoader.ts` 体量非常大，但它的角色很清楚：

它不是某个插件的 loader，而是整个插件生态的资源发现、校验、合并、降级、缓存和来源治理工厂。

如果说 `src/utils/plugins/loadPluginCommands.ts` 负责“把插件内容折叠成命令”，那这个文件负责的则是：

- 插件从哪里来
- 哪些插件应该生效
- 插件目录里有哪些组件
- manifest 和 marketplace 信息怎样合并
- 出错时怎样降级继续

14.2 它最值得先看的一点，是插件来源优先级被写得非常明确

这个文件里 `mergePluginSources(...)` 和 `loadAllPlugins()/loadAllPluginsCacheOnly()` 一起，把来源优先级写得非常清楚：

1. session-only plugins (`--plugin-dir`)
2. marketplace-based plugins
3. built-in plugins

同时还有一个非常产品化的例外：

- managed settings 锁定的插件优先于 session override

也就是说，作者在这里认真处理了三种意图冲突：

- 用户本地临时意图
- 已安装插件状态
- 管理策略意图

这不是“扫目录”的思路，而是“多来源配置治理”的思路。

14.3 createPluginFromPath(...) 很关键：它把一个目录正式变成 LoadedPlugin

这是这个文件里最核心的建模函数之一。

它做的事情非常像插件世界里的装配流水线：

1. 读 plugin.json
2. 建 LoadedPlugin
3. 自动探测 commands/ 、 agents/ 、 skills/ 、 output-styles/
4. 解析 manifest 里的 commands/agents/skills
5. 校验路径是否存在
6. 收集 hooks
7. 把错误记录下来但不中断整体加载

这说明 Claude Code 不是要求插件目录绝对标准化才能进入系统，而是尽量容忍缺项、只把真正的问题记录成错误。

这非常像成熟插件平台的做法：

- 尽量接住不完整插件
- 不让单点问题阻断全局
- 但把错误来源和组件类型记清楚

14.4 它对 manifest 与 marketplace entry 的关系处理得很成熟

这里最有意思的一块，是：

- plugin 自己可能带 plugin.json
- marketplace entry 也可能补充 commands/agents/skills/hooks/outputStyles

这个文件没有简单覆盖，而是区分：

- strict
- 非 strict
- supplement
- conflict

也就是说，它没有天真地假设“只有一个真相源”，而是承认插件生态里存在：

- 插件自带声明
- 市场侧补充声明
- 两者冲突

并且认真定义了冲突语义。

这一点非常像平台治理，而不是本地工具脚本。

14.5 loadAllPlugins() 和 loadAllPluginsCacheOnly() 体现了非常清楚的启动路径分层

这两个函数非常值得夸。

它们不是简单“一个带缓存一个不带缓存”，而是在表达两种不同的产品路径：

- 全量刷新路径：允许拉最新插件
- 启动期路径：只读 cache，避免首屏被 git clone / 网络阻塞

也就是说，作者明确区分了：

- “我要最新”
- “我要快启动”

这就是成熟产品里的关键设计。

启动性能不是顺手优化，而是被单独建模成一条路径。

14.6 assemblePluginLoadResult(...) 才是真正的总装中心

这个函数把整条插件主线串起来了：

- marketplace loader
- inline --plugin-dir
- built-in plugins
- source merge
- dependency verify / demote
- settings cache

也就是说，真正的“插件结果”不是某个目录里扫出来的东西，而是一份经过：

- 多来源合并
- 依赖校验
- 启停状态整理
- 设置合并

之后的 `PluginLoadResult`。

这就是为什么我会说它更像插件供应链后端，而不是插件目录工具。

14.7 这文件的工程感很强：错误不只是抛出，而是被结构化收集

这个文件里到处都在做一件事：

- `errors.push(...)`
- 带上 `source`
- 带上 `plugin`
- 带上 `path`
- 带上 `component`

这说明插件加载在 Claude Code 里不是“失败就异常退出”，而是：

错误也是平台状态的一部分。

只有这样，后面的：

- `plugin` 管理界面
- `reload` 流程
- 调试日志
- 遥测

才有可能做得好。

14.8 这个模块最值得借鉴的地方

如果让我总结 `src/utils/plugins/pluginLoader.ts` 最值得学的点，我会给五个关键词：

1. 来源治理
2. 组件发现
3. 冲突语义
4. 启动分层
5. 错误结构化

它真正厉害的地方，是把插件系统从“读几个目录”提升成了：

一个有来源优先级、有合并规则、有缓存路径、有平台治理意识的插件装载工厂。

15. `src/commands.ts`：整个命令世界的编排台

15.1 如果 `Tool.ts` 是能力契约，那 `commands.ts` 就是命令议会

`src/commands.ts` 很容易被看成一个“导出命令数组”的地方，但它真正负责的是：

把 Claude Code 里所有不同来源、不同可见性、不同用途的命令，整理成一个可以被宿主、模型和 UI 同时消费的统一命令世界。

这里面混着很多东西：

- built-in commands
- feature-gated commands
- plugin commands
- plugin skills
- bundled skills
- builtin plugin skills
- workflow commands
- dynamic skills
- MCP skills

这已经不是静态注册表了，而是命令编排台。

15.2 `COMMANDS()` 不是常量数组，而是“延迟求值的内建命令工厂”

这一点非常关键。

它没有直接 top-level 构造全部命令，而是用 `memoize(() => [...])`，并且里面还混有：

- `feature(...)` 驱动的条件命令
- `require(...)` 做延迟加载
- auth / provider 相关分支
- internal only commands

这说明命令系统在 Claude Code 里不是“写死的菜单”，而是：

- 跟 feature flag 有关
- 跟部署形态有关
- 跟用户身份有关
- 跟启动性能有关

15.3 getSkills() 和 loadAllCommands() 这两层把“命令世界”拆得很清楚

这里的层次其实非常漂亮：

- getSkills() 负责技能来源装配
- getPluginCommands() 负责插件命令
- getWorkflowCommands() 负责工作流命令
- COMMANDS() 负责内建命令
- loadAllCommands() 把它们统一合并
- getCommands() 再做 availability / isEnabled / dynamic skills 插入

这说明作者很清楚：

- 收集来源是一层
- 合并总表是一层
- 当前用户可见命令筛选又是一层

这是成熟的 registry 写法，不是“最后 return 一个大数组”那么简单。

15.4 meetsAvailabilityRequirement() 很能体现 Claude Code 的命令不是统一可见的

这个函数体现了一个常被忽略的事实：

不是每个命令都应该对所有用户出现。

这里显式按 auth/provider 环境判断：

- claude-ai
- console

也就是说，命令系统不仅受 feature flag 影响，还受用户接入形态影响。

这是非常产品化的做法，因为某些命令的可见性本来就应由能力环境决定。

15.5 动态技能的插入位置很有讲究，这不是随便 append

getCommands() 里最有产品味的一点，是：

- 先构造 baseCommands
- 再拿 dynamicSkills
- 然后不是简单拼到尾部
- 而是插到 “plugin skills 之后、built-in commands 之前”

这很说明作者知道命令列表不仅是功能总和，还是一个认知排序空间。

也就是说，命令的顺序在这里是有语义的：

- 哪些更像扩展能力
- 哪些更像宿主内建
- 动态发现的技能应该挤进哪一层

这种排序治理非常重要。

否则命令世界会越来越乱。

15.6 `getSkillToolCommands()` 与 `getSlashCommandToolSkills()` 特别能体现“同一命令表，多种视图”

这两个函数很值得单独看。

它们说明同一个命令世界会被不同消费方以不同规则切片：

- SkillTool 需要看哪些 prompt 命令可给模型调用
- slash command tool 需要看哪些更像“技能”
- MCP skills 还要单独过滤

这说明 `commands.ts` 不只是命令注册表，还是命令视图工厂。

同一个底层命令集，被不同上层用不同过滤规则消费。

15.7 它和 cache 失效体系也是连着的

`clearCommandMemoizationCaches()` 和 `clearCommandsCache()` 这两块很关键。

它们说明命令系统不是静态生成一次就完，而是要和这些变化保持同步：

- dynamic skills loaded
- plugin reload
- skill cache clear
- skill search index clear

这也再次证明 Claude Code 的命令世界是活的，不是编译期常量。

15.8 这个模块最值得借鉴的地方

如果让我总结 `src/commands.ts` 最值得学的点，我会给五个关键词：

1. 命令编排
2. 多来源合并
3. 可见性治理
4. 多视图消费
5. 动态失效

它真正厉害的地方，是把：

- 宿主命令
- 插件命令
- 技能
- 工作流
- 动态发现能力

全都收进一个还能持续演化的命令世界里。

16. `src/remote/RemoteSessionManager.ts`：远端会话在本地宿主里的托管控制器

16.1 它和 `bridgeMain.ts` 不是一回事

`src/bridge/bridgeMain.ts` 负责的是桥接主循环；
而 `src/remote/RemoteSessionManager.ts` 负责的是：

当远端 CCR 会话已经存在时，本地宿主怎样把它接成一个可持续交互的会话端点。

它更像 `remote` 模式里的 `session adapter`。

16.2 这个文件最重要的价值，是把“收消息”和“发消息”故意拆成两条不同通道

它的设计很清楚：

- 收消息：走 `SessionsWebSocket`
- 发消息：走 `sendEventToRemoteSession(...)`

这说明远端 `session` 在 `Claude Code` 里不是对称 `socket` 模型，而是：

- 下行订阅流
- 上行事件提交

这种拆法很像真实生产系统里的会话总线，而不是 `demo` 式全双工封装。

16.3 它保留了权限流和取消流，这一点非常关键

这里会处理：

- `control_request`

- `control_cancel_request`
- `control_response`

最关键的是：

- 权限请求会进入 `pendingPermissionRequests`
- 本地侧可以 `respondToPermissionRequest(...)`
- 服务端取消时还能触发 `onPermissionCancelled(...)`

这说明 `remote session` 不是“把输出搬回来”，而是把整套协作控制面也带回来了。

16.4 它和 `SessionsWebSocket.ts` 的关系非常像“会话管理器 + 传输层”

`src/remote/SessionsWebSocket.ts` 负责：

- 建连接
- 重连
- `ping/pong`
- `close code` 处理
- `token/header` 认证

而 `src/remote/RemoteSessionManager.ts` 负责：

- 解释收到的控制消息
- 管理 `pending permission requests`
- 暴露 `sendMessage` / `cancelSession` / `respondToPermissionRequest`

这又是一次典型的分层成功：

- `transport` 留在 `transport`
- `session` 语义留在 `manager`

16.5 这个模块最值得借鉴的地方

如果让我总结 `src/remote/RemoteSessionManager.ts` 最值得学的点，我会给四个关键词：

1. 远端会话托管
2. 上下行分离
3. 权限状态回流
4. `transport` / `session` 分层

它真正厉害的地方，是让远端会话在本地看起来依旧像一条可控的 Claude 会话，而不是一个被动订阅流。

17. src/bootstrap/state.ts : 宿主启动期与会话期共享状态的总底座

17.1 这是 Claude Code 少数真正意义上的“全局内核文件”

src/bootstrap/state.ts 体量很大，而且开头就写了非常重的话：

- DO NOT ADD MORE STATE HERE
- THINK THRICE BEFORE MODIFYING

这很说明问题。

作者知道这不是普通 util，而是 Claude Code 的全局状态底座。

17.2 它和 src/state/AppStateStore.ts 是两套不同层次的 state

这一点很重要：

- AppStateStore.ts 更像运行时共享控制面，偏 UI / REPL / runtime 观察层
- bootstrap/state.ts 更像宿主启动期与会话底层全局状态

这里存的东西非常底层：

- sessionId
- cwd
- projectRoot
- inlinePlugins
- registeredHooks
- mainThreadAgentType
- isRemoteMode
- directConnectServerUrl
- invokedSkills
- prompt cache 各种 latch

所以这不是“前端 store”，而是宿主 runtime 的全局内核状态。

17.3 它最值得注意的地方，是状态不是一个大对象直接外露，而是 accessor 风格

虽然内部确实有一个巨大的 STATE，但外部访问方式是：

- getSessionId()
- setInlinePlugins()

- `getIsRemoteMode()`
- `registerHookCallbacks()`
- `addInvokedSkill()`

这种 accessor 风格很有意义：

- 至少保住了写入口
- 让语义集中在函数名里
- 给未来做校验/副作用留口子

也就是说，作者没有把它做成 Redux，也没有把它做成一堆随手可改的裸全局，而是在全局状态现实和一定程度封装之间做了折中。

17.4 这文件真正厉害的地方，是它把“启动期偶发状态”和“会话期持续状态”放进了同一个宿主语义空间

这里既有：

- `allowedSettingSources`
- `chromeFlagOverride`
- `useCoworkPlugins`
- `sessionBypassPermissionsMode`

也有：

- `telemetry counters`
- `last API request`
- `registered hooks`
- `dynamic skill preservation`
- `remote / direct connect state`

这说明 Claude Code 的宿主不是传统 CLI 那种“启动即结束”的模型，而是一个持续运行的进程宿主。所以启动态、会话态、观察态其实都要在这里贯通。

17.5 它特别像一个“会话级内核变量表”

如果用操作系统的类比去看，这文件很像：

- `session kernel vars`
- `runtime flags`
- `telemetry accumulators`
- `transient feature latches`
- `hook registries`

这也解释了为什么它必须放在 `bootstrap/` 下面。

因为它不是业务层状态，而是整台宿主机器起来之后就要依赖的基础环境。

17.6 这个模块最值得借鉴的地方

如果让我总结 `src/bootstrap/state.ts` 最值得学的点，我会给四个关键词：

1. 宿主内核状态
2. accessor 封装
3. 启动态与会话态贯通
4. 全局但不裸奔

它真正厉害的地方，是在一个不可避免要“有全局状态”的 Agent 宿主里，尽量把全局状态做得有边界、有命名、有写入口。

18. `src/services/api/claude.ts`：模型请求、流式解析与降级重试的主循环

18.1 这不是简单的 API client，而是 Claude Code 的模型请求 runtime

`src/services/api/claude.ts` 体量巨大，但角色非常清楚：

它不是“调一下 Anthropic SDK”，而是把消息、system prompt、工具定义、缓存策略、流式事件、错误回退、遥测日志全都编成一个真正可运行的模型请求主循环。

如果说 `src/QueryEngine.ts` 是会话运行时，那这个文件就是：

- 会话运行时下面的
- 模型交互运行时

18.2 它真正重要的不是某个函数，而是一整套请求分层

这个文件至少清楚地分了这些层：

- 参数准备
- message / content 规范化
- prompt cache 策略
- 流式调用
- 非流式 fallback
- 使用量累计

- TTFT / stall / timeout 观测
- 错误分类与上报

也就是说，它不是“一个 query 函数很长”，而是把模型请求所有脏活都集中在一个 API runtime 里。

18.3 queryModelWithStreaming() / queryModelWithoutStreaming() 暴露了两种用户体验路径

这两个入口非常说明产品分层：

- 需要实时反馈时，走 streaming
- 需要一次性结果时，走 non-streaming

但更关键的是，它们背后复用的是同一个主逻辑，而不是两套分裂实现。

这说明作者一直在控制“体验模式差异”和“核心请求语义”之间的耦合。

18.4 真正最厉害的，是 streaming 这条路径已经被做成了生产级状态机

从流式主循环那一大段可以看到，它在处理的不是“读 chunk 拼字符串”，而是：

- message_start
- content_block_start
- content_block_delta
- content_block_stop
- usage 累计
- tool_use / server_tool_use
- advisor 特殊路径
- stall 检测
- idle watchdog
- stream no events 检测

这说明作者踩过大量真实线上坑。

这种代码的价值不在美，而在它把复杂的流式事件世界稳稳接住了。

18.5 idle watchdog + stall telemetry 很能体现它是产品 runtime，不是 SDK demo

这里有两套很成熟的防护：

1. 长时间没 chunk 的 idle timeout watchdog
2. chunk 间隔过长的 stall telemetry

这意味着 Claude Code 对“流式调用失败”的理解不只是：

- API 报错
- 异常抛出

而是还包括：

- 流静默挂住
- 代理半死不活
- 200 成功但实际上没事件

这才是生产环境里真正可怕的问题。

18.6 streaming -> non-streaming fallback 这条链路，是这个文件工程成熟度最高的证据之一

这部分我会给很高评价。

当流式路径失败时，它不会立刻让整个 query 崩掉，而是：

- 记录 fallback 事件
- 进入 `executeNonStreamingRequest(...)`
- 继续经过 `withRetry(...)`
- 在必要时保持 request correlation

这意味着 Claude Code 的模型请求层并不把“流式”视为唯一真理，而是把它当作默认体验路径。一旦这条路径不稳定，系统仍然尽量保证结果层面可达。

18.7 它和 `src/services/api/client.ts` 组合起来，构成了“请求 runtime + provider adapter”

这一点必须一起看。

`src/services/api/client.ts` 负责的是：

- 直连 Anthropic
- Bedrock
- Foundry
- Vertex
- OAuth / API key / cloud auth
- headers / proxy / region / timeout

而 `src/services/api/claude.ts` 负责的是：

- 构造消息和请求语义
- 驱动 streaming / fallback / usage / telemetry

这两个文件配起来，层次就非常清楚：

- `client.ts` 负责连到哪个 provider
- `claude.ts` 负责怎样把 Claude Code 的会话请求跑完

18.8 这个模块最值得借鉴的地方

如果让我总结 `src/services/api/claude.ts` 最值得学的点，我会给五个关键词：

1. 请求 runtime
2. 流式状态机
3. fallback 韧性
4. 遥测内建
5. provider 分层

它真正厉害的地方，是把“调模型”从一个 SDK 调用，做成了：

一条可观测、可降级、可回退、可跨 provider、可在真实网络条件下长期运行的模型请求主循环。

19. `src/entrypoints/sdk/controlSchemas.ts`：Claude Code 控制协议的正式定义层

19.1 这不是类型文件，而是协议宪法

`src/entrypoints/sdk/controlSchemas.ts` 表面上是在写 Zod schema，但它真正做的是：

把 Claude Code 的 SDK / remote / bridge / direct connect 这整条控制面，正式定义成一套可以校验、可以演进、可以跨语言实现的协议。

所以这不是普通 types 文件，而是协议定义层。

19.2 这文件最厉害的地方，是它把“控制面”从消息流里单独建模出来了

这里显式定义了：

- `control_request`
- `control_response`
- `control_cancel_request`
- `keep_alive`

以及大量 request subtype：

- initialize
- interrupt
- can_use_tool
- set_permission_mode
- set_model
- get_context_usage
- reload_plugins
- elicitation

这说明 Claude Code 的交互协议不是“传几条消息”那么简单，而是有一整套正式控制面。

19.3 它特别像“Agent Runtime 的 RPC 合同”

从这些 schema 看，Claude Code 里的控制协议已经覆盖了：

- 初始化能力下发
- 权限审批
- 中断
- MCP 状态
- 上下文使用量
- 插件重载
- 用户输入征询

也就是说，这不是单一用途协议，而是 Claude Code runtime 对外暴露的一整套远控合同。

19.4 StdoutMessageSchema / StdinMessageSchema 特别重要

这两个 aggregate schema 很能说明问题：

- 哪些消息可以从 CLI / runtime 输出
- 哪些消息可以从外部输入进来

也就是说，作者不是只定义消息长什么样，而是连流向都定义了。

这是协议成熟度非常高的表现。

19.5 这个模块最值得借鉴的地方

如果让我总结 `src/entrypoints/sdk/controlSchemas.ts` 最值得学的点，我会给四个关键词：

1. 协议显式化
2. 控制面独立
3. Zod 校验
4. 流向定义

它真正厉害的地方，是让 Claude Code 的控制面不再只是约定俗成，而是被正式写成了机器可验证的协议。

20. src/query/stopHooks.ts：Query 子系统在“停点”上的后处理编排器

20.1 如果 QueryEngine.ts 是主会话循环，这里就是 turn 结束后的后处理总线

src/query/stopHooks.ts 很能说明 query/ 目录的价值。
它不是另一个大 query 引擎，而是在拆：

- turn 结束后要做什么
- hooks 怎样执行
- 哪些后台工作可以 fire-and-forget
- 哪些错误要阻断 continuation

所以它更像 query 子系统里的“停点编排器”。

20.2 它最厉害的地方，是把 hook 执行当成 query 语义的一部分，而不是边角副作用

这个文件处理的不只是 Stop hooks，还包括：

- TaskCompleted hooks
- TeammateIdle hooks
- prompt suggestion
- extract memories
- auto dream
- computer use cleanup

这说明 Claude Code 里的“一次回合结束”并不等于“模型停下了”。
回合结束之后，系统还有一整套补充处理链。

20.3 它特别像“停点事件总线”

handleStopHooks(...) 会：

- 消费 hook progress 消息
- 收集 blocking errors

- 决定是否 `preventContinuation`
- 生成 `summary message`
- 必要时插入 `attachment`
- 在 `teammate` 场景继续跑后续 `hook`

这说明这里并不是简单执行几个脚本，而是在把 `hook` 结果重新编回 Claude Code 的消息世界。

20.4 Query 子系统的价值，也正是被这种模块体现出来的

`query/` 目录现在虽然不大，但从这个文件已经能看出来，它的存在意义是：

- 把 `QueryEngine` 里那些和“turn 生命周期”强相关的逻辑拆出去
- 让 `token budget`、`stop hook`、`config`、`deps` 这些问题在 `query` 语义层单独成立

这对长期演进非常重要。

否则 `QueryEngine.ts` 会继续膨胀成一切都往里塞的巨石。

20.5 这个模块最值得借鉴的地方

如果让我总结 `src/query/stopHooks.ts` 最值得学的点，我会给四个关键词：

1. 停点编排
2. `hook` 消息化
3. `continuation` 控制
4. `query` 语义拆分

它真正厉害的地方，是把“模型这一轮结束后还要发生什么”正式做成了 `query runtime` 的一部分。

21. `src/utils/sandbox/sandbox-adapter.ts`：Claude Code 把权限规则落成操作系统级沙箱的桥接层

21.1 这不是沙箱本体，而是 Claude Code 的沙箱适配器

文件开头写得很直接：

它是包在 `@anthropic-ai/sandbox-runtime` 外面的一层 `adapter`。

这层的价值就在于：

- 上层是 Claude Code 自己的 `settings`、`permission rules`、`tool` 语义
- 下层是外部 `sandbox runtime`

两者之间必须有一层翻译器。

21.2 最重要的设计：权限规则语义和 sandbox 语义被认真区分了

这里非常成熟的一点，是作者没有偷懒地把所有路径规则都混成一种解释。

它明确区分：

- permission rule 里的路径约定
- `sandbox.filesystem.*` 里的标准路径语义

甚至专门写了：

- `resolvePathPatternForSandbox(...)`
- `resolveSandboxFilesystemPath(...)`

这说明作者知道：

“规则字符串长得像路径” 不等于 “所有路径语义都一样”。

21.3 `convertToSandboxRuntimeConfig(...)` 才是真正的桥接核心

这个函数做的，是把 Claude Code 自己的：

- `permissions.allow / deny`
- WebFetch domain 规则
- `sandbox.network`
- `sandbox.filesystem`
- `--add-dir`
- settings file 保护
- `.claude/skills` 保护

全部折叠成外部 sandbox runtime 能执行的 config。

这就是 adapter 最重要的价值：

- 上层讲的是 Claude Code 语言
- 下层讲的是 sandbox runtime 语言
- 中间要有人翻译

21.4 这里的安全工程味非常浓

这个文件里有很多明显不是“普通功能开发”能自然长出来的东西，比如：

- 永远拒绝写 settings 文件
- 额外保护 `.claude/skills`

- 防 bare repo 植入
- allowManagedDomainsOnly
- policy 锁定 sandbox 设置
- 平台支持检测与依赖检测

这说明这层不是为了“让 sandbox 能跑”，而是为了：

让 Claude Code 的权限与沙箱模型在真实攻击面下成立。

21.5 它不是一次性初始化完就结束，还会动态刷新

这里还有一条很重要的线：

- initialize(...)
- 订阅 settingsChangeDetector
- refreshConfig()
- reset()

这意味着 sandbox config 不是静态常量，而是会随着 settings 变化动态更新。

这点对交互式 Agent 特别重要。

因为权限、设置、allowlist 都可能在会话中被更新。

21.6 这个模块最值得借鉴的地方

如果让我总结 `src/utils/sandbox/sandbox-adapter.ts` 最值得学的点，我会给五个关键词：

1. 规则翻译
2. 语义分层
3. 安全收口
4. 动态刷新
5. 外部 runtime 适配

它真正厉害的地方，是把 Claude Code 自己的权限语言，认真翻译成了可以真正落在 OS 级隔离上的沙箱配置。

22. `src/services/api/client.ts`：多 provider 模型客户端的统一适配器

22.1 这不是 Anthropic SDK 的薄封装，而是 provider adapter

`src/services/api/client.ts` 的真正价值，不在“new 一个 client”，而在于：

它把 Claude Code 的模型调用统一折成一个 `getAnthropicClient(...)` 接口，同时在内部适配了多种后端。

这里至少覆盖了：

- 第一方 Anthropic API
- Bedrock
- Foundry
- Vertex

也就是说，这是一层 provider adapter，不是普通 client helper。

22.2 它最成熟的地方，是把认证、headers、proxy、region 这些 provider 差异全都收口了

这个文件处理的东西非常现实：

- OAuth token refresh
- API key headers
- Bearer token for Bedrock
- Azure AD for Foundry
- GoogleAuth for Vertex
- custom headers
- proxy fetch
- region 选择
- remote/container/session headers

这说明 Claude Code 没有假设“调用模型”只有一种网络路径。

它从一开始就在按多 provider 真实环境设计。

22.3 它和 `services/api/claude.ts` 的关系非常清楚

- `client.ts` 负责连到谁
- `claude.ts` 负责怎么发、怎么流、怎么 fallback

这两层拆得很好。

否则 provider 适配逻辑一旦混进流式状态机，代码会更难收。

22.4 这个模块最值得借鉴的地方

如果让我总结 `src/services/api/client.ts` 最值得学的点，我会给四个关键词：

1. provider 适配
2. 认证收口
3. 网络环境兼容
4. 请求头治理

它真正厉害的地方，是让上层 runtime 能把“调模型”视为统一能力，而不是一堆 provider 分支。

23. `src/remote/SessionsWebSocket.ts`：Remote 会话 transport 层

23.1 如果 `RemoteSessionManager.ts` 是会话语义层，这里就是 transport 层

`src/remote/SessionsWebSocket.ts` 非常典型地负责：

- 建 WebSocket
- 认证
- ping / pong
- close code 处理
- reconnect

也就是说，它不理解“权限请求怎么处理”，但它负责“会话链路怎么活着”。

23.2 这里最值得夸的是 close code 语义处理

它并不是一掉线就盲目重连，而是区分：

- 永久关闭码
- 4001 session not found 的短期重试
- 普通 transient close 的有限次重连

这说明作者知道：

- 有些掉线该认命

- 有些掉线是暂态
- 有些掉线是 compaction 等过程里的短窗口抖动

这种 close code 语义化处理，非常像生产 transport。

23.3 transport 层自己还带着保活逻辑

这里的 `PING_INTERVAL_MS`、`startPingInterval()`、`stopPingInterval()` 很关键。

它说明 remote 会话在 Claude Code 里不是一次性连接，而是长期订阅链路。
长期链路就必须自己管保活。

23.4 这个模块最值得借鉴的地方

如果让我总结 `src/remote/SessionsWebSocket.ts` 最值得学的点，我会给四个关键词：

1. transport 分层
2. 重连预算
3. close code 语义
4. 长连接保活

它真正厉害的地方，是把 remote mode 最底层那条 WebSocket 链路做成了真正可靠的 transport，而不是“连上就算成功”。

24. `src/query/tokenBudget.ts`：Agentic Loop 的预算控制阀

24.1 文件很小，但位置很准

`src/query/tokenBudget.ts` 代码不多，但它解决的是一个非常 Agent runtime 的问题：

一轮 agentic loop 到底该在什么时候继续，什么时候停。

这不是普通 token 统计，而是 continuation 策略。

24.2 最关键的判断不是“快到预算了”，而是“边际收益还在不在”

这里有两个关键阈值：

- `COMPLETION_THRESHOLD = 0.9`
- `DIMINISHING_THRESHOLD = 500`

也就是说，作者判断是否继续，不只看：

- 预算是否还够

还看：

- 最近增量 token 是不是已经很少
- 连续继续几轮之后是不是已经进入 diminishing returns

这很有产品感。

因为真正糟糕的不是多花一点 token，而是 agent loop 在低收益状态下继续空转。

24.3 它输出的不是布尔值，而是策略决策对象

TokenBudgetDecision 会返回：

- action: 'continue'
- 或 action: 'stop'

同时还带：

- nudgeMessage
- continuationCount
- pct
- turnTokens
- budget
- completionEvent

这说明它不是一个纯底层计算函数，而是一个能直接被 runtime 和 telemetry 消费的策略节点。

24.4 这个模块最值得借鉴的地方

如果让我总结 `src/query/tokenBudget.ts` 最值得学的点，我会给三个关键词：

1. 预算策略
2. 边际收益判断
3. continuation 控制

它真正厉害的地方，是把“什么时候该停”从一种模糊感觉，做成了一个正式的 query runtime 决策点。

25. src/entrypoints/sdk/coreSchemas.ts：SDK 序列化世界的总 Schema 底板

25.1 它不是“再来一个类型文件”，而是 Claude Code 的数据宪法

src/entrypoints/sdk/coreSchemas.ts 一上来就把自己的身份说得很清楚：

- 它是 serializable SDK data types 的 Zod schema 集合
- 它是 single source of truth
- TypeScript types 反而是从这里生成出去的

这句话分量非常重。

因为这意味着在 Claude Code 里，真正的一手定义不是：

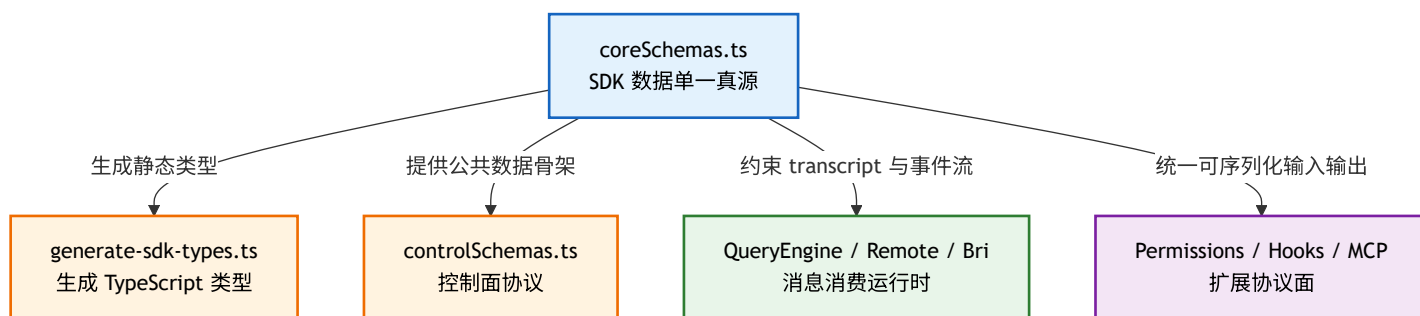
- 手写 TypeScript interface
- 再补一点 runtime 校验

而是反过来：

- 先定义可执行的 schema
- 再从 schema 生成类型
- 再让整个 SDK / remote / bridge / transcript 世界共享这套可验证的数据语言

这不是“类型更严格”这么简单。

这代表作者已经把 Claude Code 当成一个跨进程、跨宿主、跨消费方的协议系统来看待。



25.2 它真正做的是“把序列化边界正式化”

这个文件看起来像一大坨 Zod 定义，但其实内在结构很清楚，大致分成几层：

- usage / output format / config 这类基础数据
- MCP server config 与 status
- permission mode、permission update、permission result
- hook event 与各种 hook input
- SDK message、session info、result、status、stream event

换句话说，它不是在定义“某个功能模块的数据”，而是在定义：

Claude Code 哪些数据允许跨边界流动，以及这些数据应该长成什么样。

这里的“边界”至少包括：

- 主线程与 SDK host
- 本地与 remote / bridge
- query runtime 与 transcript
- 工具权限系统与 hook 系统
- MCP 接入层与宿主呈现层

所以这份 schema 文件的真正价值不在字段多，而在于它把“跨边界的数据约束”统一收口了。

25.3 lazySchema 和 placeholder 设计，说明它在认真处理真实工程问题

这个文件大量使用 lazySchema，还专门放了很多 placeholder：

- APIUserMessagePlaceholder
- APIAssistantMessagePlaceholder
- RawMessageStreamEventPlaceholder
- UUIDPlaceholder
- NonNullableUsagePlaceholder

这几个点很值得玩味。

第一，它说明 schema 之间存在相互引用或初始化顺序问题，所以作者没有把 Zod 当成“静态声明”，而是当成需要治理模块依赖的 runtime 构件。

第二，它说明 Claude Code 并没有强行把外部 SDK 类型整个拉进自己的核心协议里，而是承认有些东西：

- 来自第三方 SDK
- 或者不是纯可序列化原语
- 或者当前只需要占位语义

于是先留出协议插槽，再在边界层完成桥接。

这是一种很成熟的边界意识：

核心协议只保证自己真正能稳定拥有的那部分。

25.4 权限、Hook、MCP 都被拉进同一数据世界，这一点非常关键

看 `src/entrypoints/sdk/coreSchemas.ts` 中段，你会发现几个以前分散在不同脑图里的世界，已经在这里被统一了：

- `PermissionModeSchema`
- `PermissionUpdateSchema`
- `PermissionResultSchema`
- `HOOK_EVENTS`
- `BaseHookInputSchema`
- `McpServerStatusSchema`

这其实是在做一件非常大的事：

把“控制权”“扩展点”“外部能力状态”全部纳入统一协议层。

一旦这样做，Claude Code 后面的很多模块才能写得比较干净：

- `controlSchemas.ts` 可以专注控制面的 `request / response`
- `RemoteSessionManager.ts` 可以放心转发结构化消息
- `bridgeMain.ts` 可以把外部宿主也挂到同一消息世界上
- `hook` 系统可以不靠隐式 JSON 约定来演化

很多系统做着做着会出现：

- `hook` 是一套野生 JSON
- `permission callback` 是另一套形状
- `SDK event` 又是一套
- `remote transport` 自己再发明一套

Claude Code 明显在主动避免这种协议撕裂。

25.5 SDKMessageSchema 才是它最核心的中枢

这个文件里最有代表性的定义之一，是最后那组消息 `schema`：

- `SDKAssistantMessageSchema`
- `SDKUserMessageSchema`
- `SDKResultMessageSchema`
- `SDKSystemMessageSchema`
- `SDKPartialAssistantMessageSchema`
- `SDKAPIRetryMessageSchema`
- `SDKTaskStartedMessageSchema`
- `SDKTaskProgressMessageSchema`

- SDKToolUseSummaryMessageSchema
- SDKSessionStateChangedMessageSchema
- 最终被 SDKMessageSchema 总并

这说明 Claude Code 的 transcript / stdout / SDK 流不是“输出字符串”，而是：

一个正式定义过的事件代数。

只要这个代数成立，后面很多事都会顺：

- CLI 能渲染
- Desktop / IDE 能订阅
- Remote 能转发
- Bridge 能恢复
- SDK host 能做二次消费
- 测试也能按消息类型断言

这也解释了为什么 `src/entrypoints/sdk/controlSchemas.ts` 重要，但还不是最底层。
`controlSchemas.ts` 更像“控制面定义”；
`coreSchemas.ts` 才更接近“Claude Code 可序列化世界的物理定律”。

25.6 它的代价：文件会很大，但这是对的“大”

这个文件当然有代价：

- 很长
- 类型面极宽
- 读起来不像业务代码，像协议手册

但这种“大”不是坏味道，而是平台成熟后的必然。

因为当系统开始跨：

- CLI
- SDK
- remote
- bridge
- plugins
- hooks
- MCP

这些边界时，你要么接受一个大的协议底板，要么接受一堆互不一致的小协议碎片。

Claude Code 选的是前者。

从长期演进角度看，我认为这是更对的路线。

25.7 这个模块最值得借鉴的地方

如果只提炼一个结论，我会说：

先把“哪些数据能跨边界流动”定义成正式 schema，再去做功能扩张。

这会让系统从一开始就具备：

- 校验能力
- 类型生成能力
- 协议演进能力
- 多宿主复用能力

Claude Code 的 `coreSchemas.ts`，本质上就是它 SDK 世界的协议底盘。

26. `src/query/config.ts`：Query 子系统开始做“配置快照化”的分层信号

26.1 它不是小配置文件，而是 query runtime 的“只读前提集”

`src/query/config.ts` 很短，但我会把它看得比体量重要得多。

因为它做的事情不是“把几个环境变量搬出去”，而是明确提出了一种分层：

- `QueryConfig`：query 入口时一次性快照的不可变配置
- `State`：每轮 step 里不断变化的状态
- `ToolUseContext`：运行时工具可读写的宿主上下文

这三者一旦分开，`query.ts` 才有机会从超重 orchestration 里继续往下拆。

26.2 注释里其实已经把未来重构路线写出来了

这个文件开头的注释非常值钱，大意是：

- immutable values 在 `query()` 入口一次性 snapshot
- 把这些值从 per-iteration state 和 mutable `ToolUseContext` 中拆开
- 这样未来抽 `step()` 才更可行
- 纯 reducer 可以只吃 `(state, event, config)`

这不是普通的“顺手整理一下”。

这说明作者已经明确意识到，query 子系统内部要继续分层时，首先必须区分三种东西：

1. 真正的配置
2. 会变的运行状态
3. 带副作用的宿主能力

很多运行时系统后期难拆，就是因为这三样东西糊成一团。

`src/query/config.ts` 虽然很小，但它是在提前修路。

26.3 它刻意不收 `feature()`，这个边界意识很成熟

这个文件最有趣的点之一，是注释里明确写了：

- 这里收 runtime gates
- 但故意不收 `feature()` gates
- 因为 `feature()` 是 tree-shaking boundary
- 必须保留在 guarded block 附近，方便 dead-code elimination

这句话很能体现 Claude Code 的工程成熟度。

因为它不是单纯追求“代码整洁”，而是在兼顾：

- 架构分层
- 打包裁剪
- 初始化顺序
- 测试隔离

也就是说，作者知道“把所有开关都抽进 config 文件”在表面上很漂亮，但实际会破坏 bundling 语义。于是它只抽那些适合抽的：

- `streamingToolExecution`
- `emitToolUseSummaries`
- `isAnt`
- `fastModeEnabled`

这是一种很克制的抽象方式，我很喜欢。

26.4 `buildQueryConfig()` 的价值，在于“入口统一采样”

`buildQueryConfig()` 干的事情其实非常简单：

- 取 `sessionId`
- 读取 `statsig` gate
- 读取 `env` flag
- 返回一份 `QueryConfig`

但这恰恰是它重要的地方。

因为一旦 query 在开始执行后还不断去四处读：

- 全局状态
- 环境变量
- feature gate

那它的行为就会越来越难预测，也越来越难测试。

而这里的思路是：

在 query() 入口先把这一轮真正需要的只读前提拍一张快照。

后面整个执行过程都基于这张快照往下走。

这会极大提高：

- 可推理性
- 可测试性
- 可重放性

26.5 这个模块为什么小而重要

如果只看产出，它就是：

- 一个 QueryConfig 类型
- 一个 buildQueryConfig() 函数

但如果从架构演化角度看，它其实是一个非常强的信号：

Claude Code 的 query 子系统正在从“大函数编排”走向“配置 / 状态 / 副作用上下文”三分。

这意味着后面如果继续瘦身 src/QueryEngine.ts 和 query.ts，不是没路，而是已经开始铺设中间层了。

26.6 这个模块最值得借鉴的地方

最值得借鉴的，不是它写了什么逻辑，而是它体现出的重构方法论：

- 先抽不可变配置
- 再稳住状态机边界
- 最后才谈 reducer 化与 step 拆分

这比一上来“把大文件拆成几个 util”高级得多，也稳得多。

27. `src/services/api/withRetry.ts`：模型请求容错策略的主闸门

27.1 它不是 `retry helper`，而是 API 故障策略引擎

`src/services/api/withRetry.ts` 名字看起来很普通，但读完会发现它一点都不普通。

它不是：

- “失败了就 `sleep` 一下再试”
- “指数退避封一下”

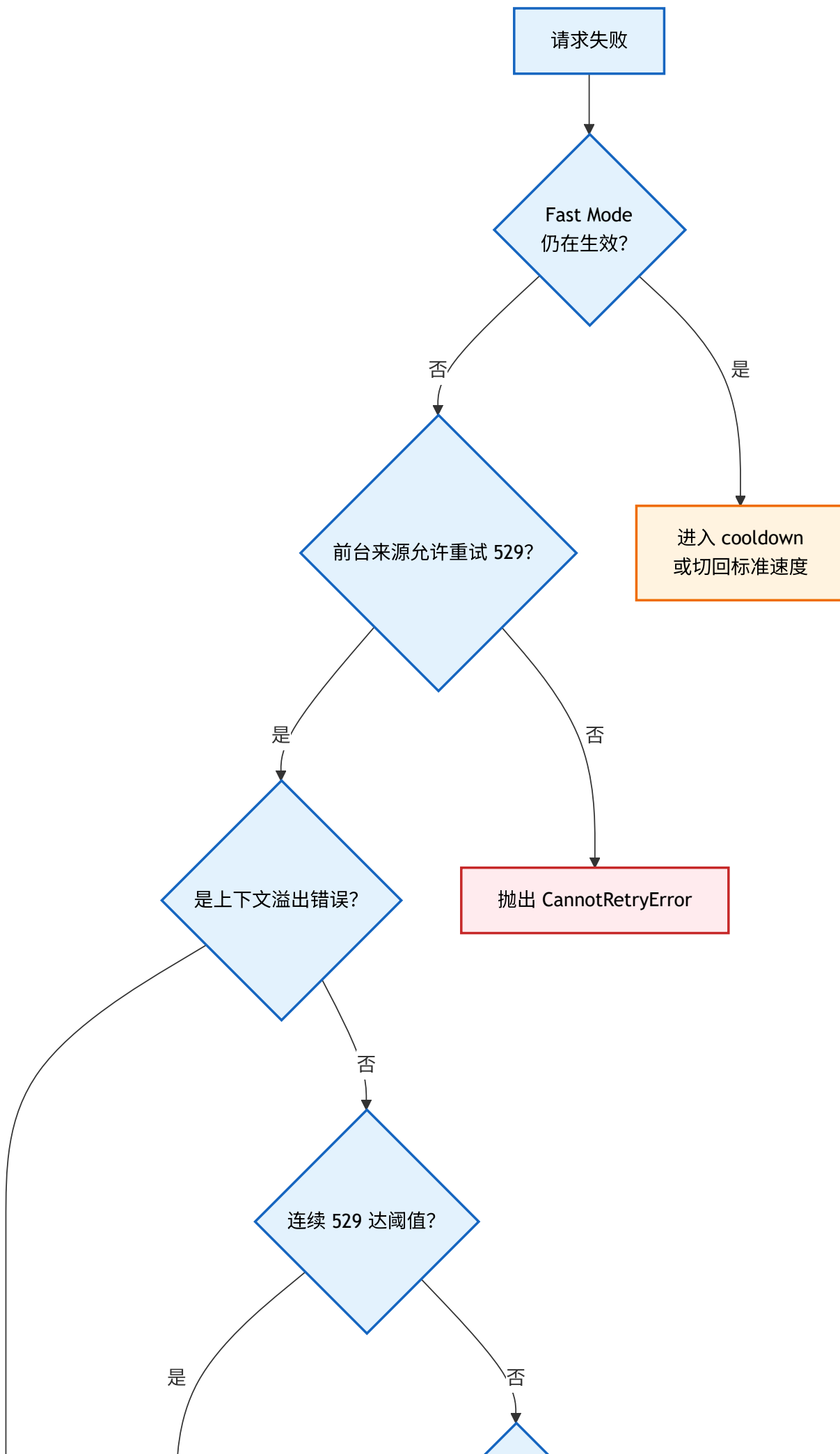
它真正做的是：

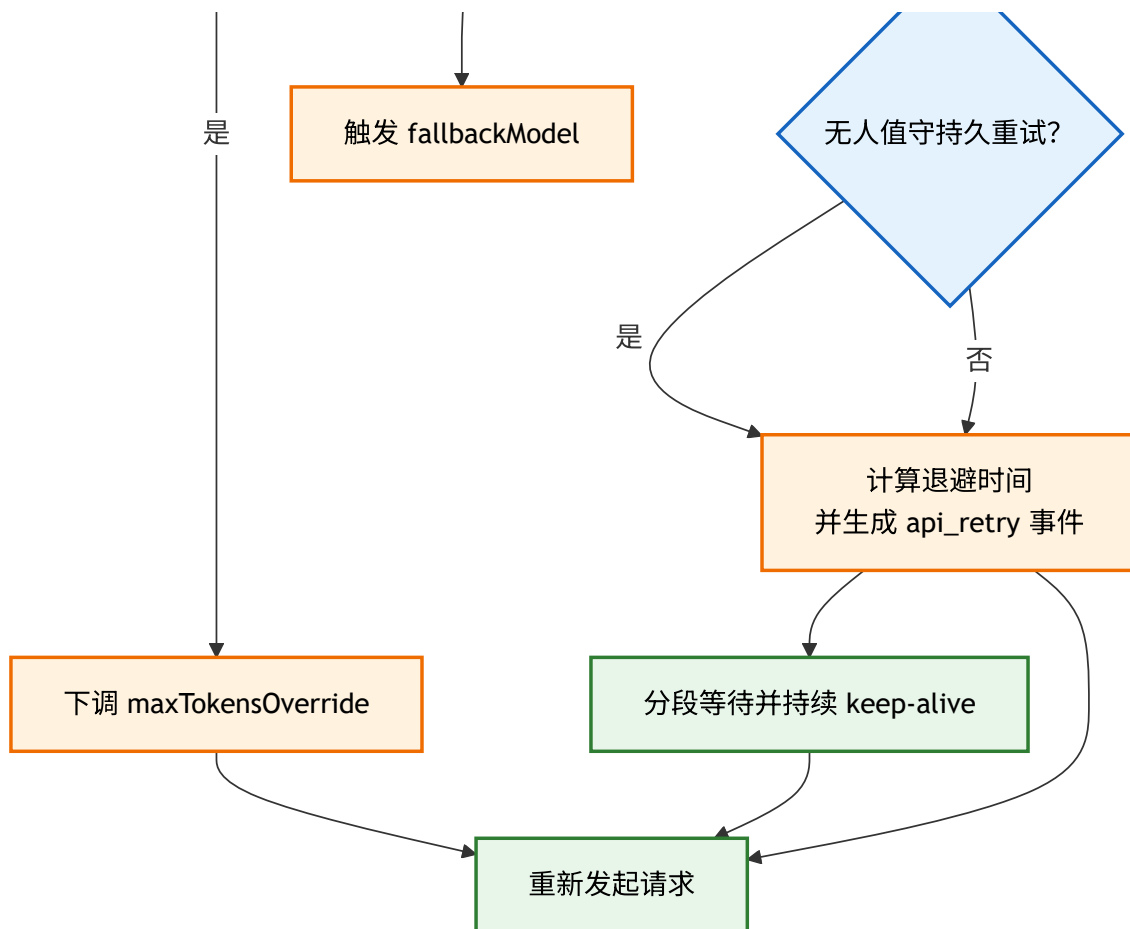
把 Claude Code 在模型请求上的容错哲学，集中实现成一个正式策略层。

这里面同时处理了：

- 普通 `retry / exponential backoff`
- 429 / 529 的差异化处理
- `foreground / background source` 的区别
- `fast mode` 的 `cooldown` 与回退
- `persistent unattended retry`
- `stale keep-alive connection` 失效
- OAuth / AWS / GCP 认证缓存刷新
- `max_tokens` 上下文溢出的自动调参
- `fallback model` 触发

这已经不是“重试工具函数”，而是一个小型故障编排器。





27.2 这个文件最成熟的点，是它不把所有失败都当成同一种失败

这里有几段判断特别能说明问题。

第一， `FOREGROUND_529_RETRY_SOURCES` 明确区分了：

- 用户正在等结果的前台请求
- 用户根本感知不到的后台请求

只有前者才值得在容量雪崩时继续重试。

后者要尽快失败，避免放大网关压力。

第二， `isPersistentRetryEnabled()` 又引入了“无人值守会话”的第三种世界：

- 普通前台会话
- 普通后台会话
- unattended 长驻会话

于是重试策略不再只是技术逻辑，而是产品语义：

- 人在等，就可以多扛一下
- 人看不见，就尽量别放大故障
- 人不在，但任务必须跑完，就进入持久重试

这是很强的系统设计意识。

27.3 fast mode、fallback、认证恢复被收进同一条故障链，非常像线上系统

这个文件最“线上”的地方，是它没有把错误处理拆成很多互相不知道彼此存在的小 util，而是把真实故障链条收在了一起。

比如：

- fast mode 遇到短 `retry-after`，直接短等后保留 cache 继续
- fast mode 遇到长等待，进入 `cooldown`，切回标准速度，避免 `cache thrashing`
- 连续 529 达阈值时，可能触发 `fallbackModel`
- 遇到 `ECONNRESET` / `EPIPE` 这种 `stale connection`，会禁用 `keep-alive`
- Bedrock / Vertex 的认证错误，会清掉各自云凭证缓存
- 401 / OAuth revoked 会强制刷新令牌

这意味着 `src/services/api/withRetry.ts` 处理的其实不是“重试次数”，而是：

一次模型调用在复杂线上环境里可能经历的故障路径图。

所以它会显得很重，但这正是因为它吸收了真实世界的复杂度。

27.4 它用 AsyncGenerator 往外吐系统消息，这一点非常漂亮

`withRetry()` 最值得夸的设计之一，是它不是简单返回一个 `Promise<T>`，而是：

- `AsyncGenerator<SystemAPIErrorMessage, T>`

这意味着在等待重试期间，它可以持续向外产出：

- `api_retry`
- `keep-alive` 风格的 `retry` 提示

于是上层 runtime 不会只知道“还没返回”，而是能把这段等待重新编进 Claude Code 的消息流。

这件事非常关键，因为 Claude Code 是产品，不是库。

当一次请求卡在重试期时，宿主需要的是：

- 可观测
- 可呈现
- 可中断

而不是“内部默默 `sleep 30 秒`”。

所以这个模块跟 `src/services/api/claude.ts` 的关系也很清楚：

- `claude.ts` 负责请求主循环

- `withRetry.ts` 负责失败路径的正式治理

27.5 `CannotRetryError` 和 `FallbackTriggeredError` 让失败也有结构

这里还做了两个很重要的错误对象：

- `CannotRetryError`
- `FallbackTriggeredError`

这说明作者不满足于“抛一个 `Error` 字符串出去”，而是希望上层能够知道：

- 这是彻底不能再试了
- 还是需要切模型
- 当前 `retry context` 是什么

这种结构化失败语义，会让上层：

- 更容易判断 UI 呈现
- 更容易做 `telemetry`
- 更容易在不同 `provider / mode` 下复用容错策略

这就是典型的平台化写法。

27.6 它的代价：文件会越来越像事故手册

这个文件当然也有代价。

随着 `provider`、`mode`、`retry` 语义、认证方式越来越多，它会不断吸入：

- 特例
- 条件分支
- 环境判断
- `header` 语义
- `mode` 语义

所以它天然会越来越像“线上故障经验总表”。

但我反而觉得这类复杂度不该被假装不存在。

Claude Code 至少做对了一件事：

把复杂容错集中在一处治理，而不是让每条请求路径各自发明半套 `retry` 规则。

27.7 这个模块最值得借鉴的地方

如果让我总结 `src/services/api/withRetry.ts` 最值得学的地方，我会给四个词：

1. 故障分型
2. 策略集中
3. 重试可见
4. 失败结构化

它真正厉害的地方，是把“请求失败怎么办”这件事，从零散经验，提升成了 Claude Code 的正式 runtime policy。

28. 二十七个模块拼起来，才是 Claude Code 的真正主线

如果把这二十七个文件连起来看，Claude Code 的核心路径就更清楚了：

28.1 main.tsx

负责把整个宿主环境启动起来：

- 配置
- 模式
- 权限
- 插件
- 技能
- MCP
- REPL / server / remote

28.2 Tool.ts

定义模型能调用哪些能力，以及这些能力怎样被：

- 验证
- 授权
- 调用
- 呈现
- 追踪

28.3 QueryEngine.ts

把一次次用户消息组织成一个长期会话，把：

- 消息流
- system prompt
- memory
- usage
- transcript
- query loop

统一进一个 session runtime。

28.4 AppStateStore.ts

当整个系统需要共享一份运行时状态面时，它负责：

- 定义状态形状
- 定义默认宿主状态
- 提供统一 store 原语
- 让 UI 与 runtime 共用同一总线

28.5 useCanUseTool.tsx

当系统准备真正落地一次工具调用时，它负责决定：

- 这次权限能否直接放行
- 该先跑自动检查还是先弹窗
- 是否要交给 bridge / channel / swarm leader
- 用户拒绝后怎样把结果反馈给模型

28.6 client.ts

当系统需要接入外部能力源时，它负责：

- 建立 MCP 连接
- 将 MCP server 转译成 Tool / Command / Resource
- 处理认证、掉线、重连与结果裁剪
- 把外部协议行为折叠进宿主运行时

28.7 AgentTool.tsx

当模型决定把某段工作交给别的 agent 时，它负责判断：

- 起哪个 agent
- 用什么执行形态
- 是否后台化
- 是否远程化

- 是否 worktree 隔离
- 结果如何重新编码回主线程

28.8 LocalAgentTask.tsx

当系统最终决定把工作外包给子 agent 时，它把这件事变成一个：

- 可并发
- 可跟踪
- 可通知
- 可取消

的任务对象。

28.9 bridgeMain.ts

当 Claude Code 需要被远端控制、恢复和持续托管时，它负责：

- 注册 bridge environment
- 轮询 work queue
- 孵化本地 session
- 管理 heartbeat、resume、shutdown 与 worktree cleanup

28.10 createDirectConnectSession.ts

当 Claude Code 需要把一个外部 direct-connect server 会话接进来时，它负责：

- 发起 /sessions 握手
- 传递工作目录与权限初始化语义
- 用 schema 校验 server 返回
- 折叠成内部 DirectConnectConfig
- 把后续 WebSocket 会话交给连接层继续托管

28.11 loadPluginCommands.ts

当 Claude Code 需要把插件里的 Markdown 命令、技能目录、manifest metadata 接进主宿主时，它负责：

- 发现并整理插件 command / skill 内容
- 处理命令命名空间与去重
- 解析 frontmatter、模型、工具权限、参数与 shell 配置
- 做变量替换与用户配置注入
- 折叠成宿主统一 Command 协议并并入命令总表

28.12 loadSkillsDir.ts

当 Claude Code 需要把本地、托管、项目、动态发现、条件激活这些技能统一接进来时，它负责：

- 解析 skill frontmatter
- 折叠成统一 Command
- 去重与条件分流
- 动态发现并激活相关技能
- 把技能系统接入宿主命令世界

28.13 directConnectManager.ts

当 Claude Code 需要维持一条 direct connect 远端会话时，它负责：

- 建立 WebSocket 会话
- 分流内容消息与控制消息
- 转交权限请求
- 回写控制响应与 interrupt
- 把远端托管会话变成宿主可消费的流式 runtime

28.14 pluginLoader.ts

当 Claude Code 需要把多个来源的插件资源整理成统一插件结果时，它负责：

- 发现 marketplace / session / builtin 插件
- 解析 manifest 与目录组件
- 合并 marketplace entry 与 plugin.json
- 处理 cache-only 与 full-load 两条启动路径
- 结构化收集错误并生成 PluginLoadResult

28.15 commands.ts

当 Claude Code 需要把内建命令、插件命令、技能、工作流和动态能力并成一个命令世界时，它负责：

- 组织多来源命令装配
- 处理 availability / isEnabled 筛选
- 插入 dynamic skills
- 生成 SkillTool / slash skill 等不同视图
- 维护命令缓存与失效机制

28.16 RemoteSessionManager.ts

当 Claude Code 需要把一个远端 CCR 会话接进本地宿主时，它负责：

- 托管订阅连接
- 转交 SDK 消息
- 管理权限请求与取消
- 通过 HTTP 发送用户事件
- 把远端 session 折成宿主可控的交互会话

28.17 bootstrap/state.ts

当 Claude Code 需要在整个宿主生命周期里维持一批全局底层状态时，它负责：

- 存放 session / cwd / projectRoot 等核心宿主变量
- 保存插件、hook、skill、telemetry、cache latch 等底层状态
- 提供统一 accessor 读写入口
- 让启动期和会话期共享同一套内核变量表

28.18 services/api/claude.ts

当 Claude Code 需要真正把一次会话请求发给模型并稳定拿回结果时，它负责：

- 组织消息、system prompt、工具与缓存参数
- 驱动流式请求
- 解析流式事件并累计 usage
- 监控 stall / idle timeout
- 在需要时回退到 non-streaming + retry 路径

28.19 controlSchemas.ts

当 Claude Code 需要把 SDK、remote、bridge、direct connect 这些控制交互正式协议化时，它负责：

- 定义 control request / response / cancel / keep_alive
- 定义 initialize、permission、interrupt、reloadPlugins 等 request subtype
- 用 Zod 做协议校验
- 区分 stdin / stdout 两个消息方向

28.20 query/stopHooks.ts

当 Claude Code 需要在一次 turn 结束后继续运行 hooks、后台任务和 continuation 判定时，它负责：

- 执行 Stop / TaskCompleted / TeammateIdle hooks

- 汇总 hook 结果与错误
- 决定是否阻断继续运行
- 把停点后处理重新编回消息流

28.21 sandbox-adapter.ts

当 Claude Code 需要把自己的权限规则真正落成操作系统级沙箱约束时，它负责：

- 翻译 permission rules 与 sandbox settings
- 生成 sandbox runtime config
- 做平台、依赖、策略锁定与安全保护
- 包装实际命令执行并支持动态刷新

28.22 services/api/client.ts

当 Claude Code 需要把不同 provider 的模型接入统一成同一客户端入口时，它负责：

- 处理 OAuth / API key / cloud auth
- 适配 Anthropic / Bedrock / Foundry / Vertex
- 统一 headers / proxy / region / timeout 等网络参数
- 为上层请求 runtime 提供统一 client

28.23 SessionsWebSocket.ts

当 Claude Code 需要维持 remote mode 最底层的长连接 transport 时，它负责：

- 建立并认证 WebSocket
- 处理 ping / pong
- 管理 close code 与重连预算
- 提供 control request / response 发送原语

28.24 query/tokenBudget.ts

当 Claude Code 需要决定 agentic loop 是否继续推进时，它负责：

- 跟踪 continuation 次数与 token 变化
- 判断是否接近预算上限
- 判断是否进入 diminishing returns
- 输出 continue / stop 决策与对应提示

28.25 coreSchemas.ts

当 Claude Code 需要把 SDK、hook、permission、MCP、transcript 这些跨边界数据统一成单一真源时，它负责：

- 定义 serializable schema
- 生成可复用 TypeScript 类型
- 约束 SDK message 与 result 世界
- 为 control plane 和各宿主提供共同数据底板

28.26 query/config.ts

当 Claude Code 需要让 query runtime 朝着可拆的状态机继续演进时，它负责：

- 在 query() 入口快照不可变配置
- 分离 config、state、ToolUseContext
- 收纳 env / statsig runtime gates
- 为 step 提纯和 reducer 化预留结构

28.27 withRetry.ts

当 Claude Code 需要把模型请求失败后的重试、退避、fallback 与持久等待正式化时，它负责：

- 区分前台 / 后台 / 无人值守重试语义
- 处理 429 / 529、认证恢复、stale connection
- 处理 fast mode cooldown 与 fallbackModel
- 在等待时向宿主持持续产出 api_retry 事件

所以真正的主线不是“用户 -> 模型 -> 工具 -> 返回”。

真正的主线更接近：

宿主启动 -> 会话运行 -> 状态总线 -> 工具契约 -> 权限编排 -> MCP 接入 -> 插件命令折叠 -> 技能装配 -> Agent 分流 -> 任务协同 -> 桥接托管 -> Direct Connect 握手 -> Direct Connect 流控 -> 插件装载治理 -> 命令世界编排 -> 远端会话托管 -> 启动态内核状态 -> 模型请求主循环 -> SDK 数据骨架 -> 控制协议定义 -> Query 配置快照 -> 停点后处理 -> 沙箱执行边界 -> provider 适配 -> 重试与退避策略 -> remote transport -> token 预算控制 -> 结构化回流。

这就是为什么我会说 Claude Code 更像一个 Agent Runtime，而不是一个会调模型的终端程序。

29. 模块级评价

src/main.tsx

评价：系统总装层，工程能力强，复杂度也最重。

优点是启动、模式、入口、初始化全都统一。

风险是后续继续膨胀会变成超级巨石。

src/Tool.ts

评价：平台契约层，非常成熟。

它把 Tool 做成了完整运行时组件，而不只是函数。

src/QueryEngine.ts

评价：会话控制器，方向完全正确。

它证明作者理解 Agent 的本质是 session runtime。

问题主要是体量大，不是抽象错。

src/tasks/LocalAgentTask/LocalAgentTask.tsx

评价：多 agent 产品化能力的核心证据。

它说明 Claude Code 做的是任务调度式多 agent，而不是递归式多 agent。

src/tools/AgentTool/AgentTool.tsx

评价：多 Agent 路由与调度层，非常关键。

它把“生成子 agent”从一个普通工具动作提升成了运行时分流决策。

如果没有这一层，LocalAgentTask 再强，也只是任务容器，而不是完整的多 Agent 入口。

src/hooks/useCanUseTool.tsx

评价：权限编排层，产品味非常浓。

它最厉害的地方不是安全规则本身，而是把自动检查、人工审批、远端审批、swarm 授权统一进一套可竞争的决策流程。

这说明 Claude Code 的权限系统不是附属功能，而是运行时的一部分。

src/services/mcp/client.ts

评价：外部能力总线适配层，工程成熟度很高。

它把连接、认证、转译、容错、结果压平全都收在一起，体量很重，但方向是对的。

如果没有这一层，Claude Code 的 MCP 只能算“外挂能力”；有了这一层，它才真正变成宿主的一部分。

src/state/AppStateStore.ts

评价：运行时状态骨架，重要性被低估。

它不是最耀眼的模块，但它决定了前面那些复杂能力最后是否能收敛到一个统一控制面里。

如果没有这一层，Claude Code 会很快退化成一堆局部状态和隐式耦合。

src/bridge/bridgeMain.ts

评价：远端控制主循环，工程味非常重。

它把 Remote Control 做成了真正可持续运行的 session 托管层，而不是“把本地界面投到远端”。这也是 Claude Code 能从本地工具走向跨设备连续体验的关键模块。

src/server/createDirectConnectSession.ts

评价：服务端会话握手层，文件很薄，但边界切得很好。

它真正的价值不在“做了多少逻辑”，而在于把 HTTP 建连、schema 校验、错误语义、内部 config 折叠收成了一个稳定入口。

如果没有这一层，directConnectManager 只能直接背协议细节，上层 runtime 也很难保持干净。

src/utils/plugins/loadPluginCommands.ts

评价：插件命令协议适配层，生态意味非常重。

它把 Markdown、frontmatter、manifest metadata、插件变量、用户配置和 shell 求值都收束成统一 Command 对象，这使插件系统没有撕裂宿主命令体系。

复杂度确实高，但这是“生态接入复杂度”，不是随意堆逻辑。

src/skills/loadSkillsDir.ts

评价：技能系统总装层，产品感和运行时感都非常强。

它把 skill 从“目录里的 Markdown”提升成了可去重、可条件激活、可动态发现、可统一回收到的命令协议中的正式能力层。

如果没有这一层，Claude Code 的技能系统会很快碎成多个来源、多个格式、多个触发时机的拼盘。

src/server/directConnectManager.ts

评价：Direct Connect 流式会话控制器，虽然文件不长，但协议价值很高。

它把内容流、权限流、中断流三类交互统一在一条 WebSocket 控制面里，让远端托管会话仍然服从 Claude Code 的本地宿治理。

如果没有这一层，createDirectConnectSession.ts 拿到的 ws_url 只是连接信息，不是可持续运行的会话。

src/utils/plugins/pluginLoader.ts

评价：插件装载工厂，是整个插件生态的基础设施中枢。

它真正解决的不是“插件目录怎么读”，而是插件来源优先级、manifest 冲突、cache 路径、组件发现、错误收集这些平台级问题。

体量很大，但大得有道理，因为插件生态治理本来就是重模块。

src/commands.ts

评价：命令世界编排台，系统级价值被严重低估。

它把内建命令、插件命令、技能、工作流、动态能力和不同可见性规则统一装配成一个活的命令表，再按不同消费方切出不同视图。

如果没有这一层，Claude Code 的命令系统会迅速退化成多个互不一致的注册表。

src/remote/RemoteSessionManager.ts

评价：远端会话托管层，产品上非常关键。

它把远端会话的订阅流、发送流、权限回路和取消回路统一进一个本地 manager，让 CCR 会话在本地依然像可治理的 Claude 会话。

如果没有这一层，remote mode 只能做成被动观看，而很难做成真正可协作会话。

src/bootstrap/state.ts

评价：宿主内核状态底座，工程地位非常高。

它不是最优雅的文件，但它承担了整个进程级宿主变量表的职责，并且尽量通过 accessor 把全局状态收在可命名、可控的边界里。

这类模块不耀眼，却直接决定系统能否稳定演进。

src/services/api/claude.ts

评价：模型请求 runtime，重量级核心模块。

它把流式调用、fallback、usage 累积、缓存策略、工具 schema、错误处理和遥测日志全都编进一条 API 主循环里，明显是踩过大量真实线上问题后长出来的代码。

如果没有这一层，Claude Code 的“调模型”能力只会停留在 SDK 演示级别。

src/entrypoints/sdk/controlSchemas.ts

评价：控制协议定义层，重要性非常高。

它让 Claude Code 的 control plane 不再停留在隐式约定，而是被正式编码成可校验、可跨语言消费、可持续演进的一套 schema。

这种模块平时不显眼，但它决定了整个 SDK / remote / bridge 生态能不能稳。

src/query/stopHooks.ts

评价：query 子系统里的停点编排层，位置很关键。

它说明 QueryEngine 之外已经开始长出更细的 turn 生命周期子模块，而且这些子模块处理的是 continuation、hook 总结、后台后处理这类真正复杂的回合语义。

这是 QueryEngine 后续能否继续瘦身的好信号。

src/utils/sandbox/sandbox-adapter.ts

评价：沙箱桥接层，安全价值极高。

它把 Claude Code 自己的权限与设置语言认真翻译成 sandbox-runtime 可执行的 OS 级限制，并额外补上 settings、skills、git bare repo 等 Claude 特有攻击面保护。

这是少数明显带着安全工程气质的核心模块。

src/services/api/client.ts

评价：provider 客户端适配层，基础但非常关键。

它把不同云厂商和认证体系的复杂性吸收在一处，使上层的 `claude.ts` 可以专注于请求 runtime，而不是到处散落 provider 分支。

这类模块做不好，整个多 provider 支持会立刻变脆。

src/remote/SessionsWebSocket.ts

评价：remote transport 层，可靠性价值很高。

它对 close code、保活、重连预算、临时 session-not-found 窗口的处理都很像线上系统，而不是普通 ws helper。

如果没有这一层，remote 会话的稳定性会明显下降。

src/query/tokenBudget.ts

评价：小而准的策略模块。

它把 agentic loop 什么时候继续、什么时候停，从经验判断做成了正式的预算控制阀。

这类文件不大，却往往是系统从“能跑”走向“会收敛”的关键。

src/entrypoints/sdk/coreSchemas.ts

评价：SDK 数据契约底板，平台意味很强。

它把 message、permission、hook、MCP status、session metadata 这些跨边界数据统一成可执行 schema，再向类型系统反向生成。

如果没有这一层，Claude Code 的 SDK 世界很容易碎成多套隐式 JSON 约定。

src/query/config.ts

评价：小而关键的分层信号。

它说明 query 子系统已经开始认真区分 config、state 和 side-effect context，这对后续瘦身 `query.ts` 非常重要。

虽然文件很短，但它代表的是一条正确的重构路线。

src/services/api/withRetry.ts

评价：容错策略引擎，成熟度很高。

它把 retry、backoff、fallback、fast mode cooldown、持久重试、认证恢复这些失败路径统一收口，并且让重试过程对宿主可见。

这类模块往往最能体现一套系统到底有没有线上经验。

30. 最后的判断

如果让我用一句话总结这篇模块级拆解，我会说：

Claude Code 这套系统最厉害的地方，是它把“模型能力”放进了宿主、会话、状态、协议、权限、接入、插件折叠、技能装配、调度、任务、桥接、握手、流控、装载治理、命令编排、远端托管、内核状态、模型请求、SDK 数据骨架、控制协议、Query 配置快照、停点编排、沙箱边界、provider 适配、重试与退避策略、remote transport、token 预算二十七层结构里。

这二十七层一旦成立，Agent 才会像产品。

否则再强的模型，也只是一个会说话的命令执行器。

从模块成熟度来看，我目前最欣赏的排序是：

1. Tool.ts
2. QueryEngine.ts
3. bridgeMain.ts
4. AppStateStore.ts
5. client.ts
6. AgentTool.tsx
7. useCanUseTool.tsx
8. createDirectConnectSession.ts
9. loadPluginCommands.ts
10. withRetry.ts
11. loadSkillsDir.ts
12. commands.ts
13. services/api/claude.ts
14. coreSchemas.ts
15. sandbox-adapter.ts
16. controlSchemas.ts
17. directConnectManager.ts
18. RemoteSessionManager.ts
19. pluginLoader.ts

- 20. bootstrap/state.ts
- 21. query/stopHooks.ts
- 22. query/config.ts
- 23. services/api/client.ts
- 24. SessionsWebSocket.ts
- 25. query/tokenBudget.ts
- 26. LocalAgentTask.tsx
- 27. main.tsx

原因不是 `main.tsx` 不重要，而是它更像复杂度集散地；
而前面二十六个文件更像 Claude Code 真正的架构内核。